

COMP3026E Distributed Systems

3. Time

James Jianqiao Yu

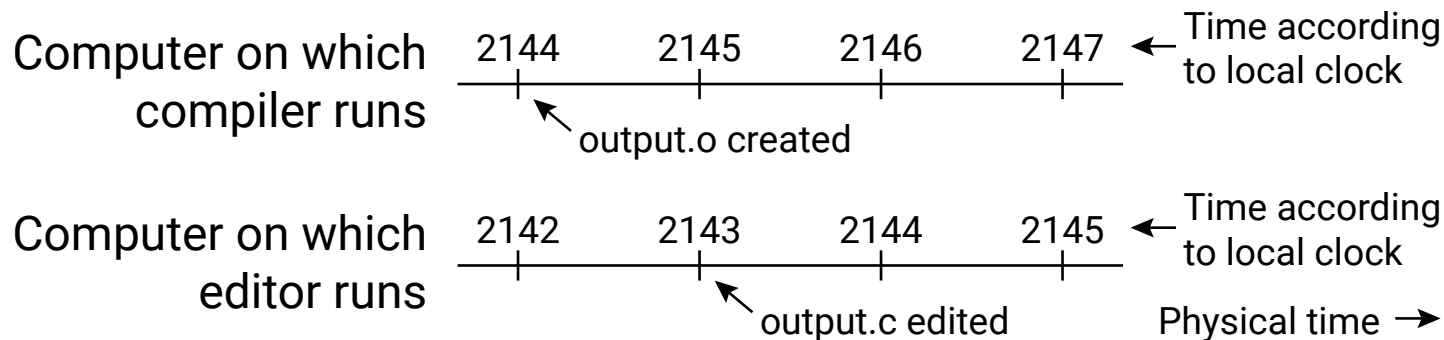
Harbin Institute of Technology, Shenzhen

Today

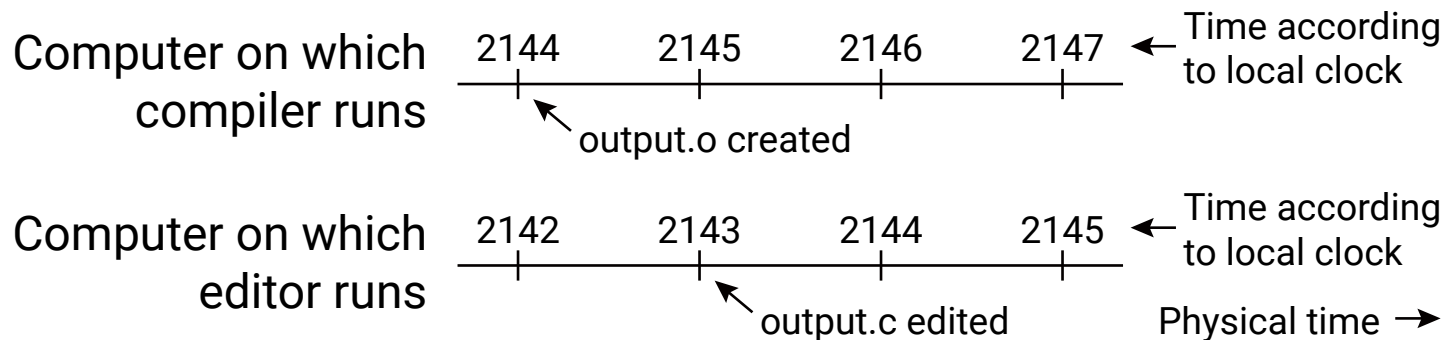


1. The need for time synchronization
2. “Wall clock time” synchronization
3. Logical Time: Lamport Clocks
4. Vector Clocks

A Distributed Edit-Compile Workflow



A Distributed Edit-Compile Workflow



- $2143 < 2144 \rightarrow$ **make** does not call compiler!

Lack of time synchronization result: **a possible object file mismatch**

What Makes Time Synchronization Hard?

- Quartz oscillator sensitive to temperature, age, vibration, radiation
 - Accuracy ~one part per million (one second of clock drift over 12 days)
- The Internet is:
 - Asynchronous: arbitrary message delays
 - Best-effort: messages don't always arrive



Just Use Coordinated Universal Time?



- UTC is broadcast from radio stations on land and satellite (e.g., the Global Positioning System)
 - Computers with receivers can synchronize their clocks with these timing signals



Just Use Coordinated Universal Time?



- UTC is broadcast from radio stations on land and satellite (e.g., the Global Positioning System)
 - Computers with receivers can synchronize their clocks with these timing signals
- Signals from land-based stations are accurate to about **0.1–10 milliseconds**
- Signals from GPS are accurate to about **one microsecond**

Just Use Coordinated Universal Time?

- UTC is broadcast from radio stations on land and satellite (e.g., the Global Positioning System)
 - Computers with receivers can synchronize their clocks with these timing signals
- Signals from land-based stations are accurate to about **0.1–10 milliseconds**
- Signals from GPS are accurate to about **one microsecond**
 - (Why don't we just put GPS receivers on all our computers?)

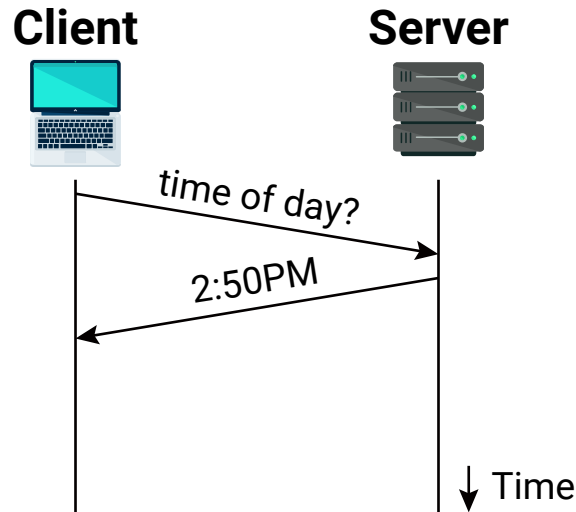
Just Use Coordinated Universal Time?

- UTC is broadcast from radio stations on land and satellite (e.g., the Global Positioning System)
 - Computers with receivers can synchronize their clocks with these timing signals
- Signals from land-based stations are accurate to about **0.1–10 milliseconds**
- Signals from GPS are accurate to about **one microsecond**
 - (Why don't we just put GPS receivers on all our computers?)

GPS signals are not available indoors, and GPS receivers are expensive

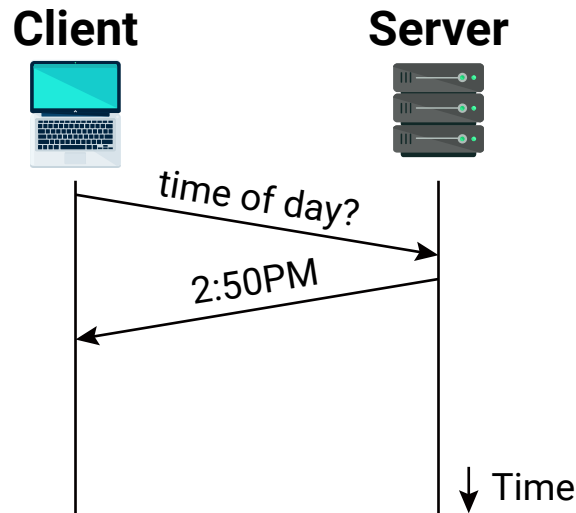
Synchronization to a Time Server

- Suppose a server with an accurate clock (e.g., GPS-receiver)
 - Could simply issue an RPC to obtain the time:



Synchronization to a Time Server

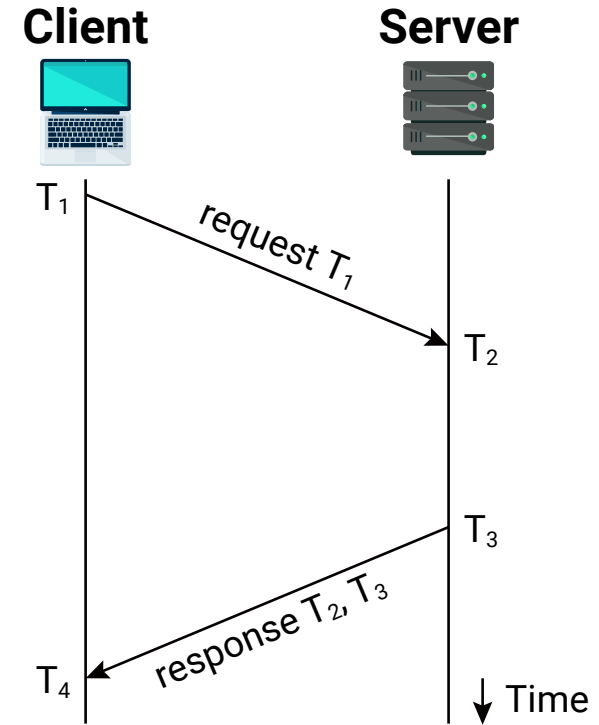
- Suppose a server with an accurate clock (e.g., GPS-receiver)
 - Could simply issue an RPC to obtain the time:



- But this doesn't account for **network latency**
 - Message delays will have **outdated** server's answer

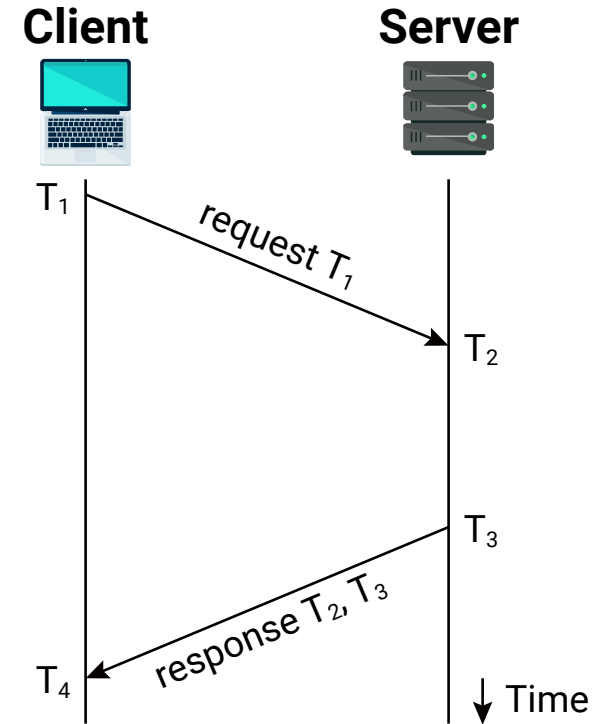
Cristian's Algorithm: Outline

1. Client sends a **request** packet, timestamped with its local clock T_1



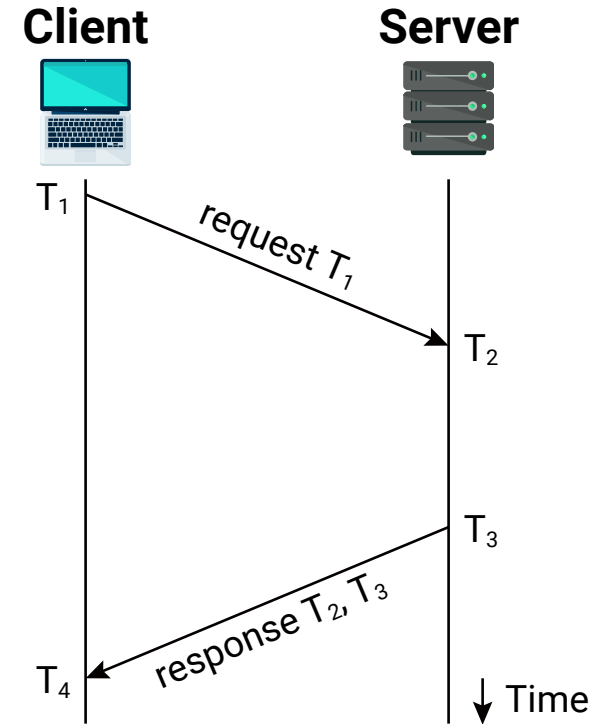
Cristian's Algorithm: Outline

1. Client sends a **request** packet, timestamped with its local clock T_1
2. Server timestamps its receipt of the request T_2 with its local clock



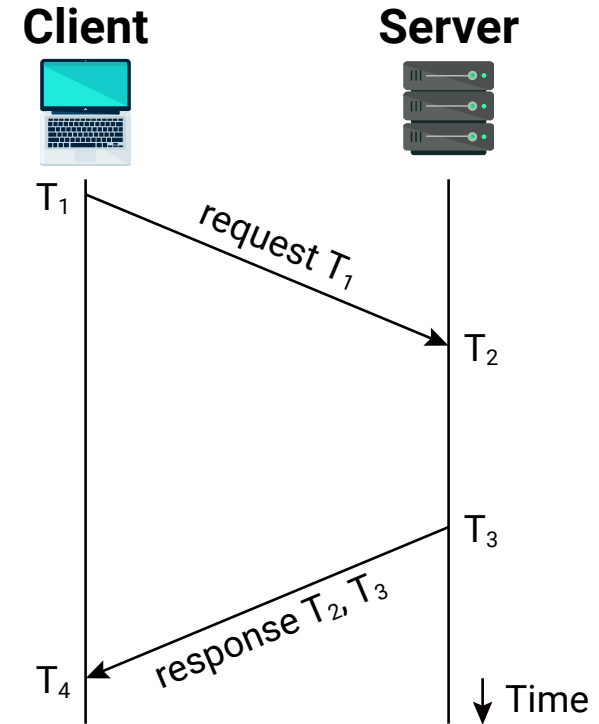
Cristian's Algorithm: Outline

1. Client sends a **request** packet, timestamped with its local clock T_1
2. Server timestamps its receipt of the request T_2 with its local clock
3. Server sends a **response** packet with its local clock T_3 and T_2



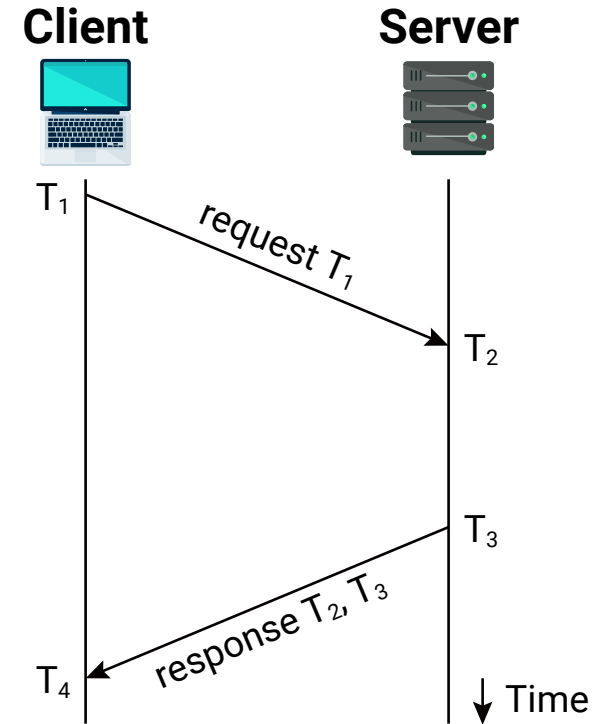
Cristian's Algorithm: Outline

1. Client sends a **request** packet, timestamped with its local clock T_1
2. Server timestamps its receipt of the request T_2 with its local clock
3. Server sends a **response** packet with its local clock T_3 and T_2
4. Client locally timestamps its receipt of the server's response T_4



Cristian's Algorithm: Outline

1. Client sends a **request** packet, timestamped with its local clock T_1
2. Server timestamps its receipt of the request T_2 with its local clock
3. Server sends a **response** packet with its local clock T_3 and T_2
4. Client locally timestamps its receipt of the server's response T_4



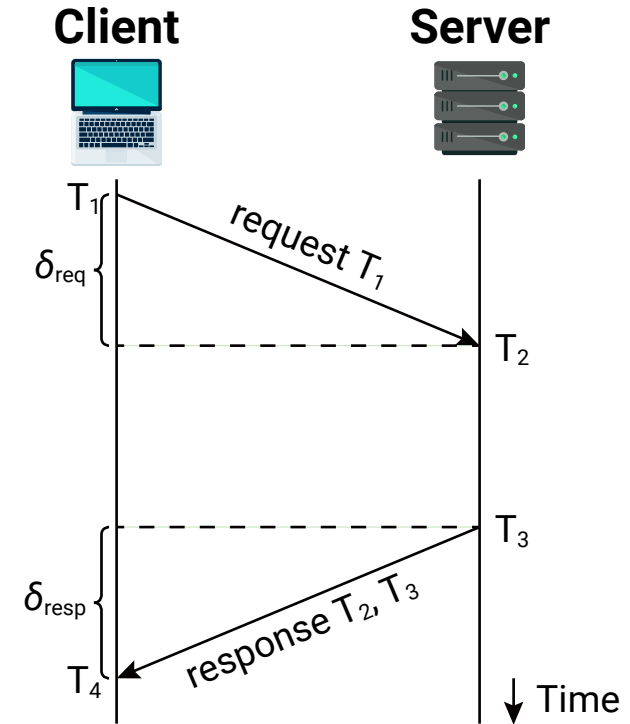
Question How can the client use these timestamps to synchronize its local clock to the server's local clock?



Cristian's Algorithm: Offset Sample Calculation



Goal Client sets clock $\leftarrow T_3 + \delta_{\text{resp}}$

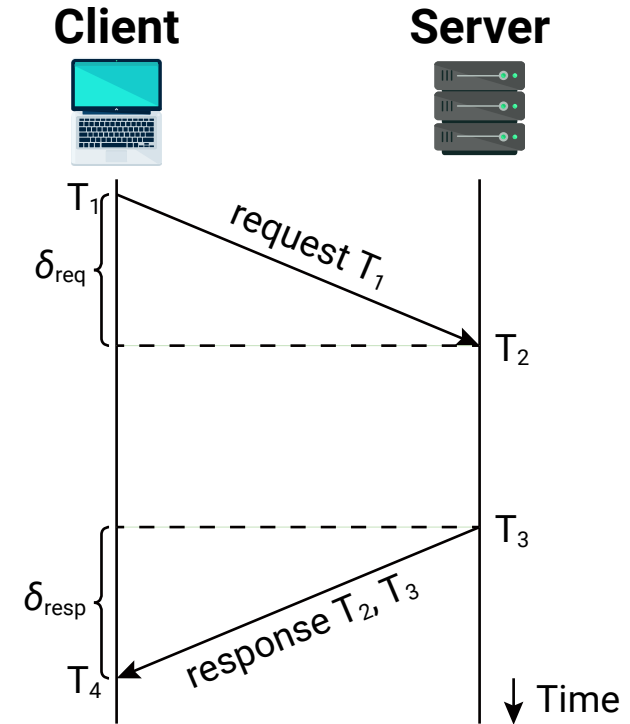


Cristian's Algorithm: Offset Sample Calculation



Goal Client sets clock $\leftarrow T_3 + \delta_{\text{resp}}$

- Client samples round trip time (RTT) δ
 - $\delta = \delta_{\text{req}} + \delta_{\text{resp}}(T_4 - T_1) - (T_3 - T_2)$
- But client knows δ , not δ_{resp}**



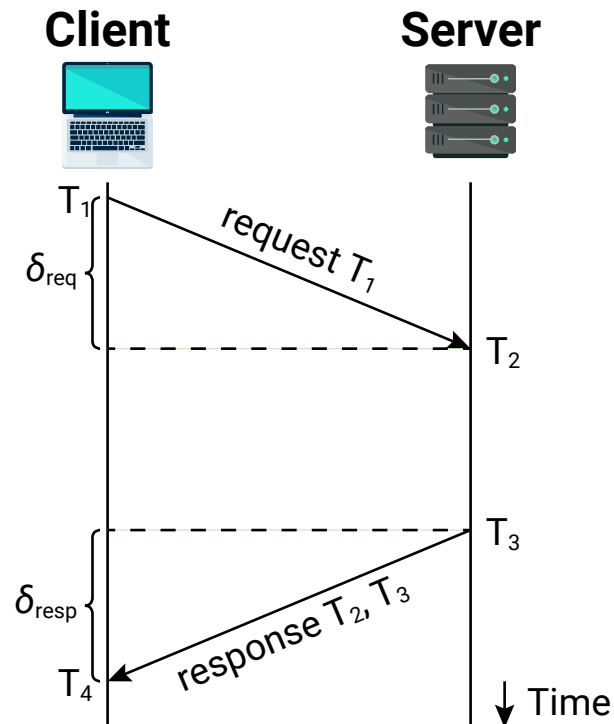
Cristian's Algorithm: Offset Sample Calculation



Goal Client sets clock $\leftarrow T_3 + \delta_{\text{resp}}$

- Client samples round trip time (RTT) δ
 - $\delta = \delta_{\text{req}} + \delta_{\text{resp}}(T_4 - T_1) - (T_3 - T_2)$
- But client knows δ , not δ_{resp}**

Assume $\delta_{\text{req}} \approx \delta_{\text{resp}}$



Cristian's Algorithm: Offset Sample Calculation

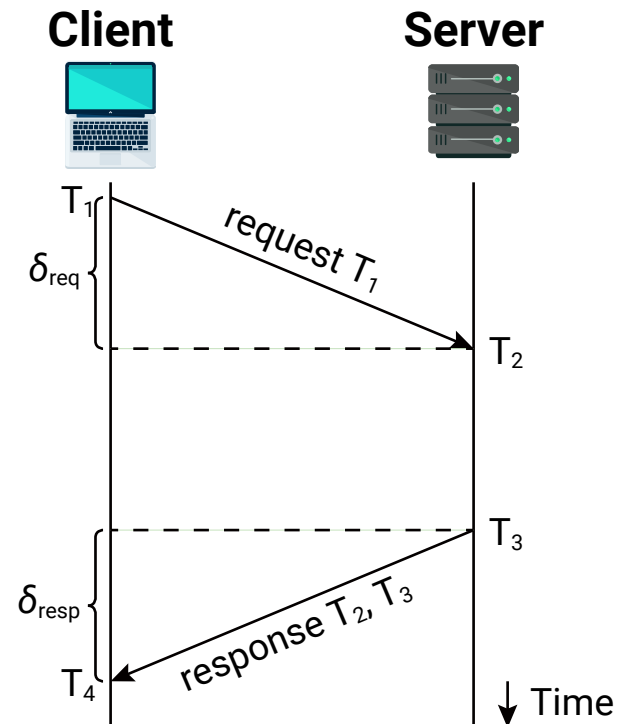


Goal Client sets clock $\leftarrow T_3 + \delta_{\text{resp}}$

- Client samples round trip time (RTT) δ
 - $\delta = \delta_{\text{req}} + \delta_{\text{resp}}(T_4 - T_1) - (T_3 - T_2)$
- But client knows δ , not δ_{resp}**

Assume $\delta_{\text{req}} \approx \delta_{\text{resp}}$

Solution Client sets clock $\leftarrow T_3 + \frac{1}{2}\delta$





NTP Clock Synchronization



- Every machine has its own clock, and they drift over time (even good hardware clocks do).
- NTP organizes machines into **a hierarchy of time servers** (from highly accurate sources like GPS/atomic clocks down to ordinary servers).
- A client periodically exchanges timestamped messages with one or more NTP servers.
- From the **round-trip delay and time offset** in these messages, the client estimates the correct current time.
- The client then **smoothly adjusts** its local clock (usually by slightly speeding up or slowing down time) instead of making sudden jumps.

Motivation: Logical Time

- A New York-based bank wants to make its transaction ledger database resilient to whole-site failures



Motivation: Logical Time

- A New York-based bank wants to make its transaction ledger database resilient to whole-site failures
- Replicate the database, **keep one copy** in SF, one in NYC



The Consequences of Concurrent Updates

- Replicate the database, keep one copy in SF, one in NYC
 - Client sends reads to the nearest copy
 - Client sends update to both copies



The Consequences of Concurrent Updates

- Replicate the database, keep one copy in SF, one in NYC
 - Client sends reads to the nearest copy
 - Client sends update to both copies



The Consequences of Concurrent Updates

- Replicate the database, keep one copy in SF, one in NYC
 - Client sends reads to the nearest copy
 - Client sends update to both copies



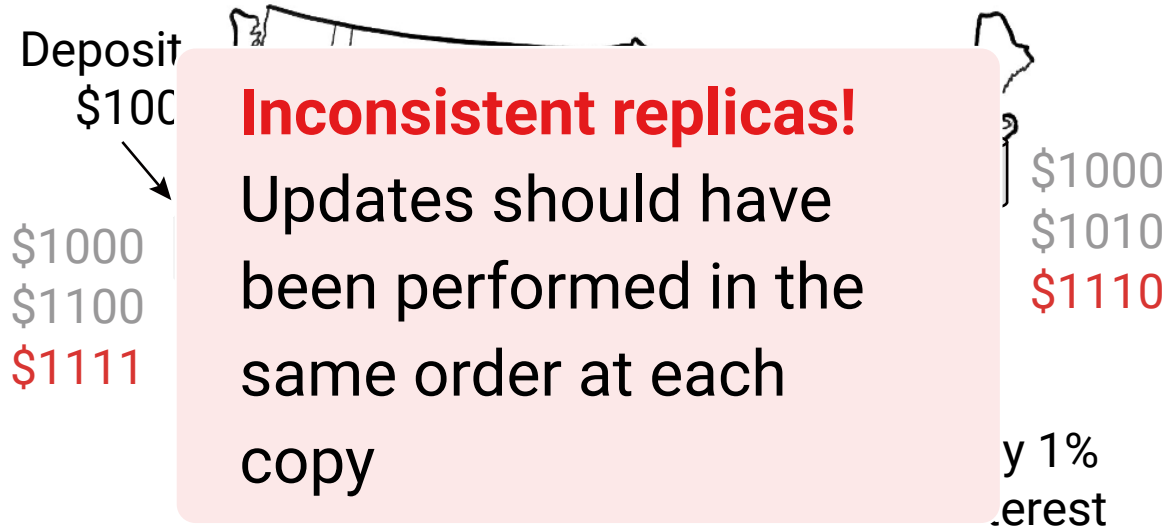
The Consequences of Concurrent Updates

- Replicate the database, keep one copy in SF, one in NYC
 - Client sends reads to the nearest copy
 - Client sends update to both copies



The Consequences of Concurrent Updates

- Replicate the database, keep one copy in SF, one in NYC
 - Client sends reads to the nearest copy
 - Client sends update to both copies





RFC 677 “The Maintenance of Duplicate Databases”



RFC 677 To the extent that the communication paths can be made **reliable**, and the clocks used by the processes kept close to **synchrony**, the probability of seemingly strange behavior can be made very small. However, the distributed nature of the system dictates that this probability can never be zero.

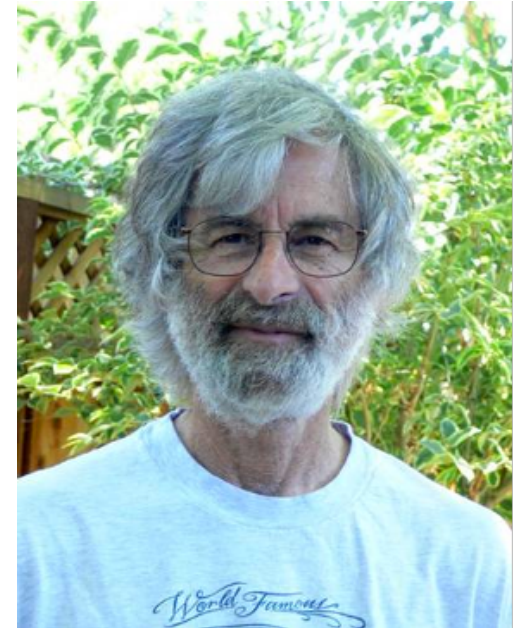
Idea: Logical Clocks

- Landmark 1978 paper by Leslie Lamport

Insight Only the **events themselves** matter

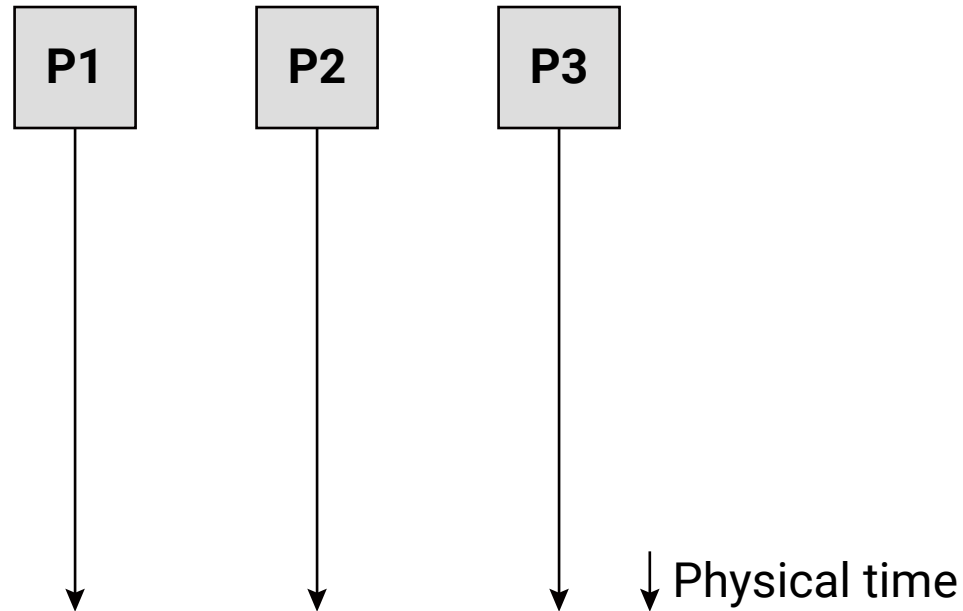
Idea Disregard the precise clock time

Instead, capture **just** a “happens before” relationship between a pair of events



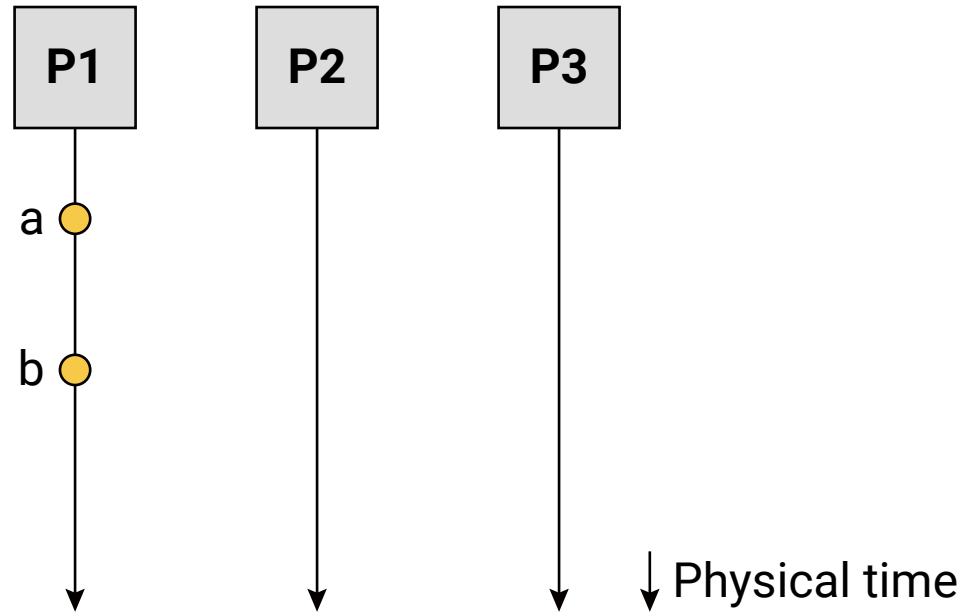
Defining “Happens-Before” ($\rightarrow$)

- Consider three processes: P1, P2, and P3
- Notation: Event a happens before event b ($a \rightarrow b$)



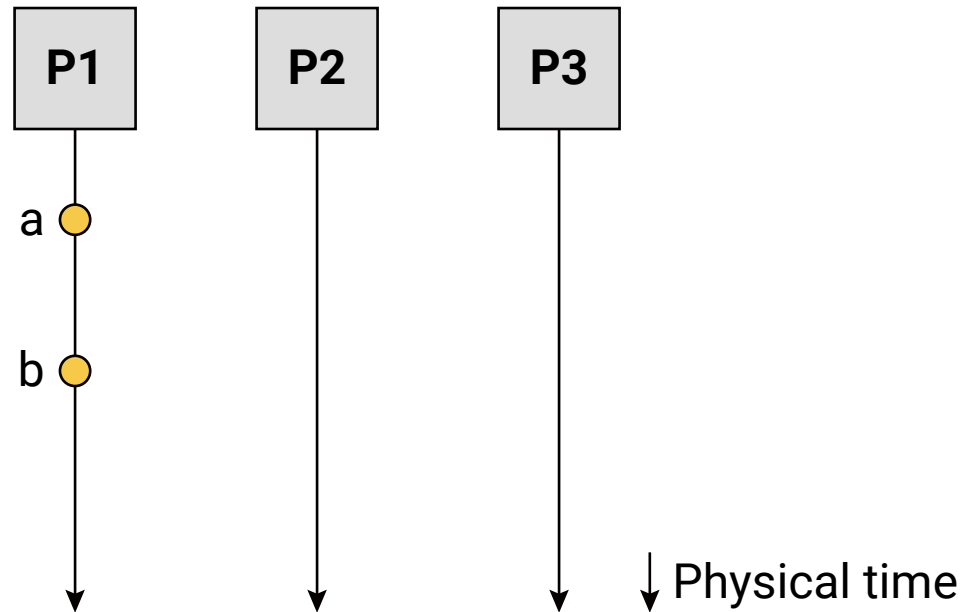
Defining “Happens-Before” ($\rightarrow$)

- Can observe event order at a single process



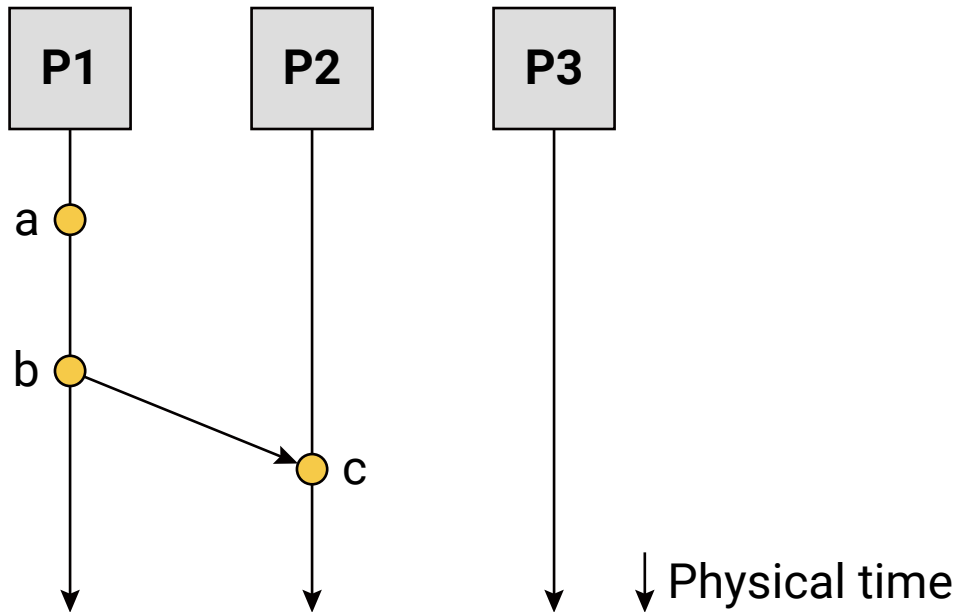
Defining “Happens-Before” ($\rightarrow$)

1. If **same process** and a occurs before b , then $a \rightarrow b$



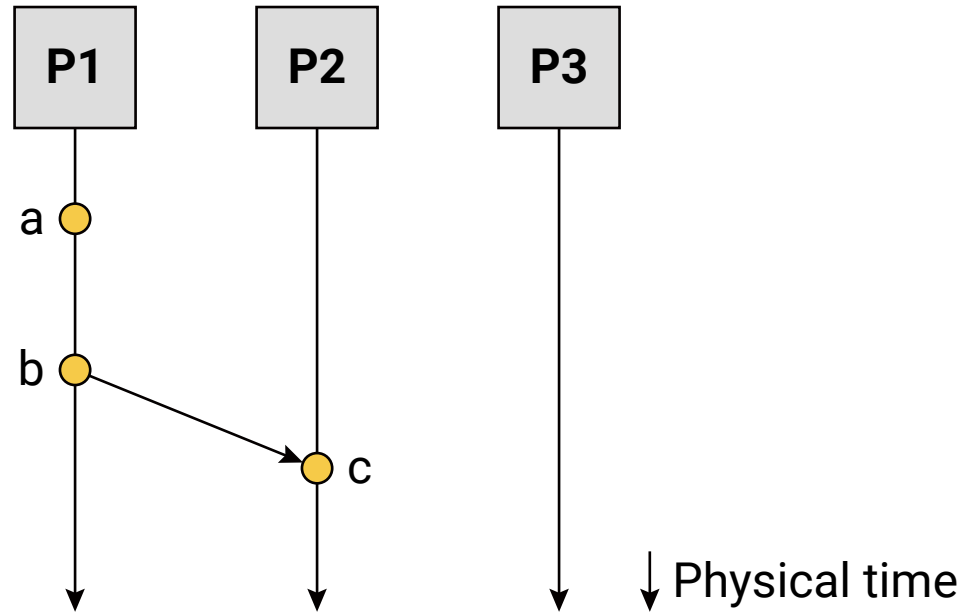
Defining “Happens-Before” ($\rightarrow$)

1. If same process and a occurs before b , then $a \rightarrow b$
2. Can observe ordering when processes communicate



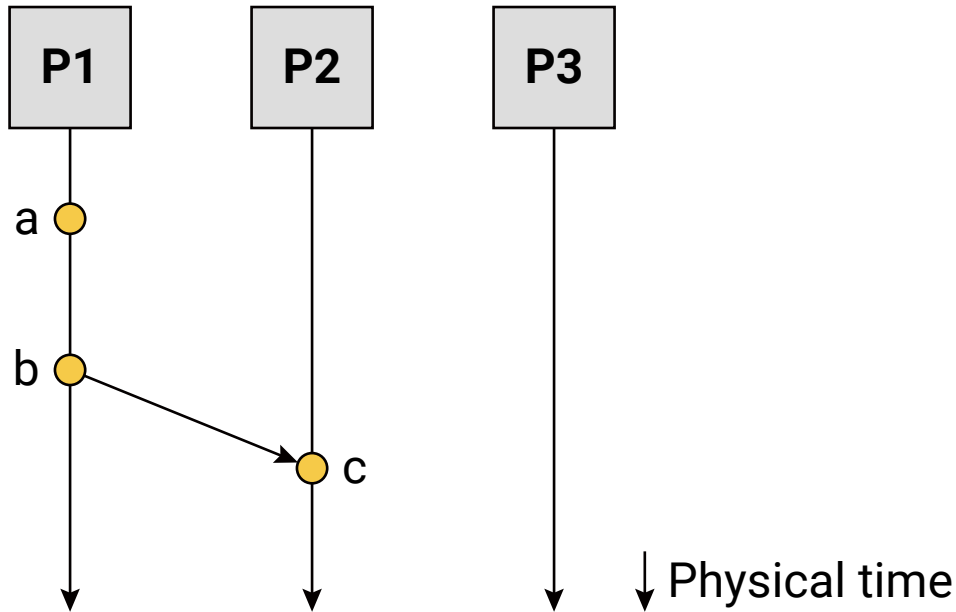
Defining “Happens-Before” ($\rightarrow$)

1. If same process and a occurs before b , then $a \rightarrow b$
2. If c is a message receipt of b , then $b \rightarrow c$



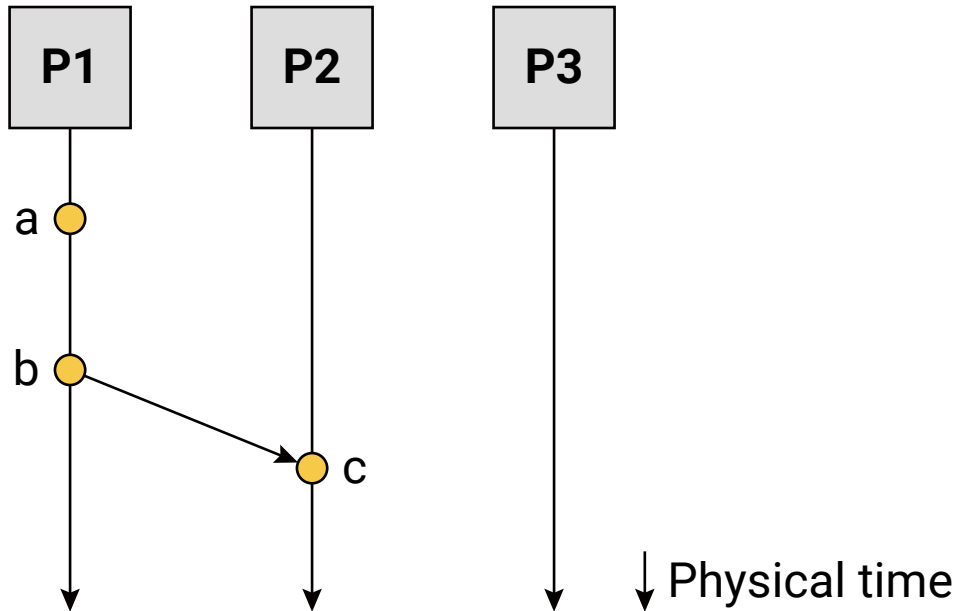
Defining “Happens-Before” ($\rightarrow$)

1. If same process and a occurs before b , then $a \rightarrow b$
2. If c is a message receipt of b , then $b \rightarrow c$
3. Can observe ordering transitively



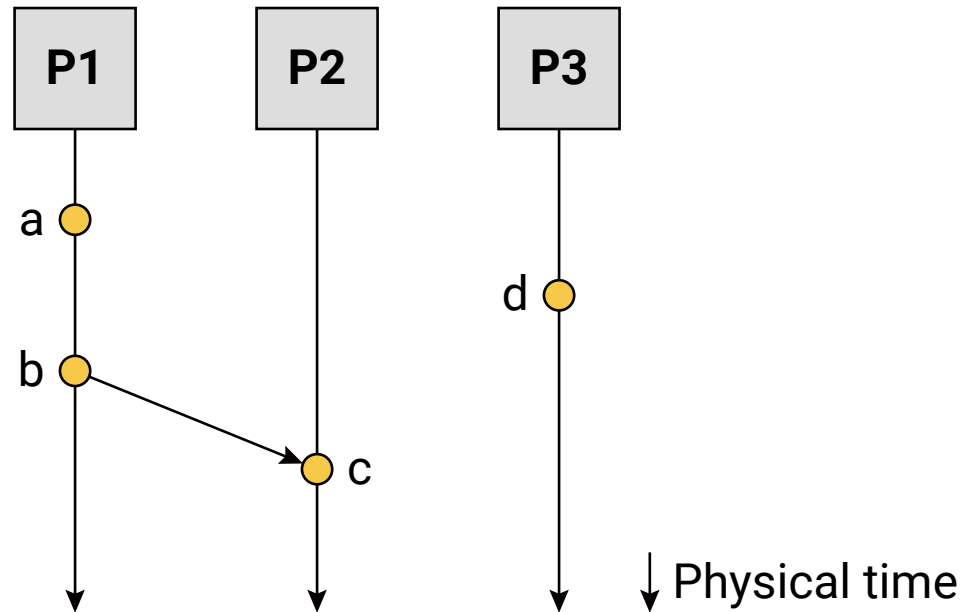
Defining “Happens-Before” ($\rightarrow$)

1. If same process and a occurs before b , then $a \rightarrow b$
2. If c is a message receipt of b , then $b \rightarrow c$
3. If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$



Concurrent Events

- Not all events are related by $\rightarrow$
- a, d not related by $\rightarrow$ so concurrent, written as $a \parallel d$



Lamport Clocks: Objective

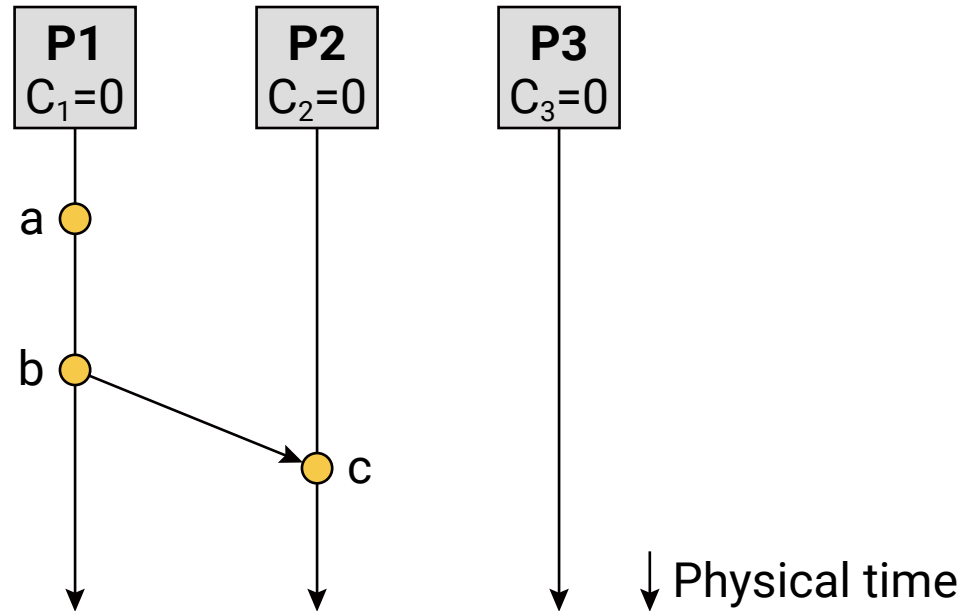
- We seek a clock time $C(a)$ for every event a

Plan Tag events with clock times; use clock times to make distributed system correct

- Clock condition: If $a \rightarrow b$, then $C(a) < C(b)$

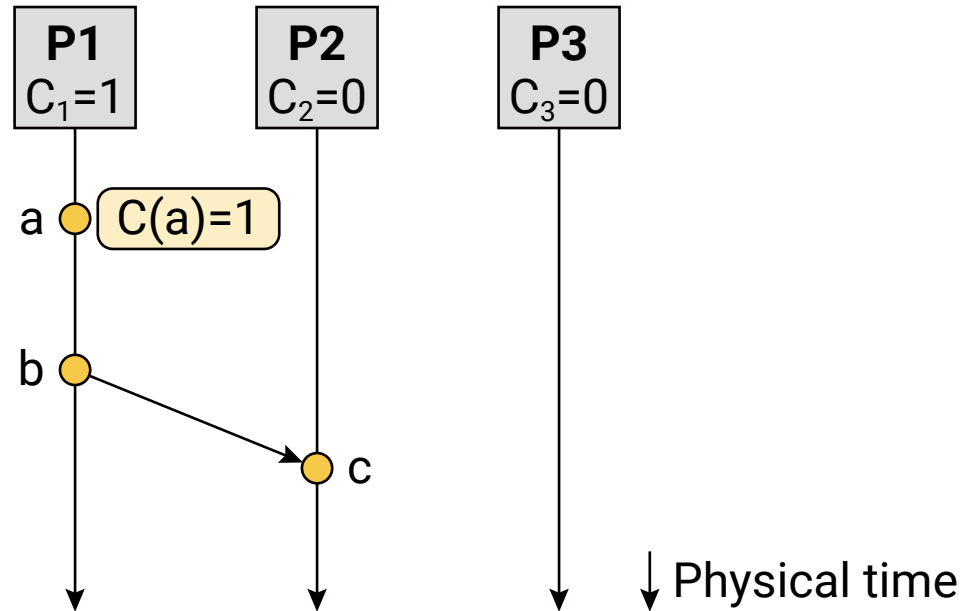
The Lamport Clock Algorithm

- Each process P_i maintains a local clock C_i
- Before executing an event, $C_i \leftarrow C_i + 1$



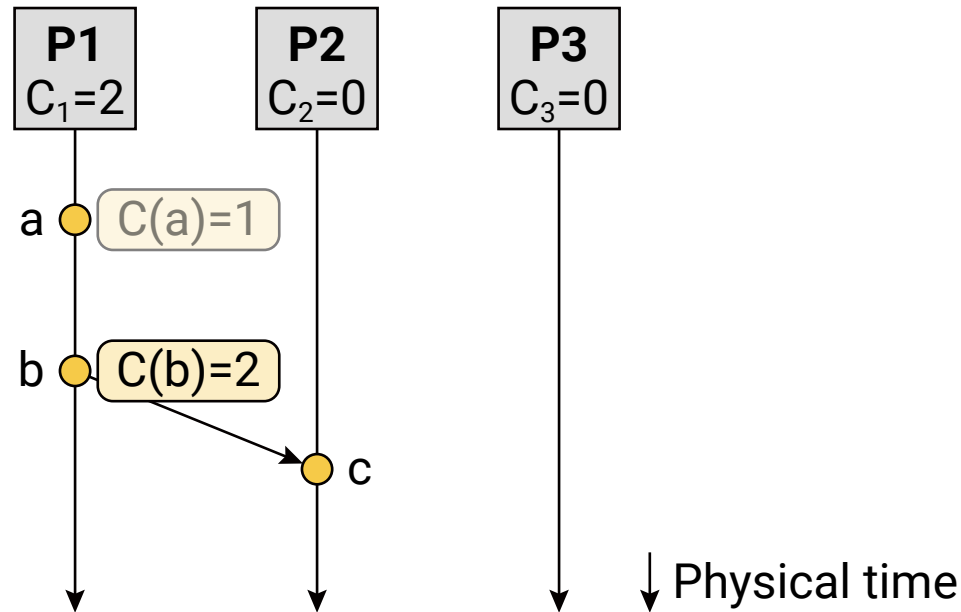
The Lamport Clock Algorithm

- Each process P_i maintains a local clock C_i
- Before executing an event, $C_i \leftarrow C_i + 1$
 - Set event time $C(a) \leftarrow C_i$



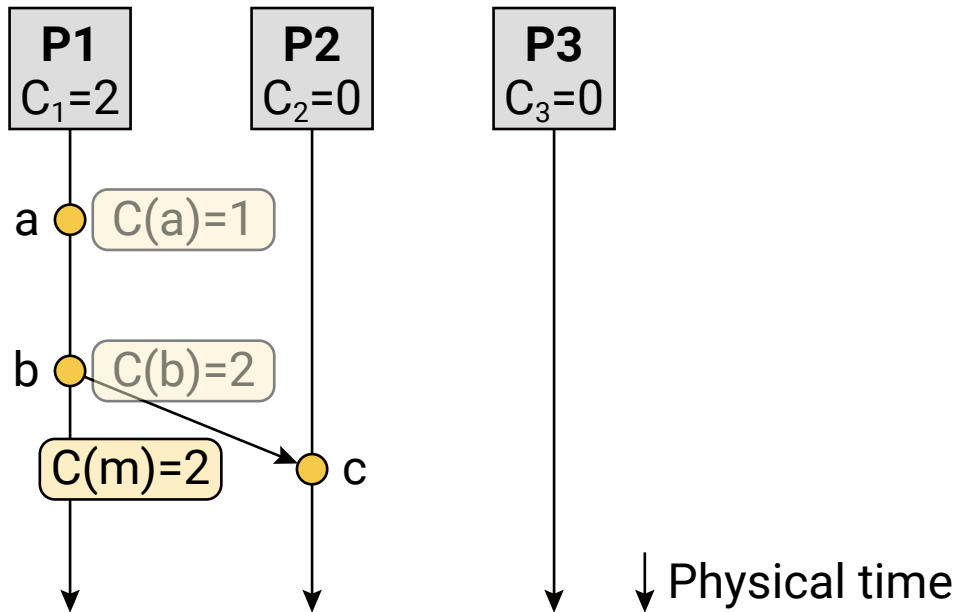
The Lamport Clock Algorithm

- Each process P_i maintains a local clock C_i
- Before executing an event, $C_i \leftarrow C_i + 1$
 - Set event time $C(b) \leftarrow C_i$



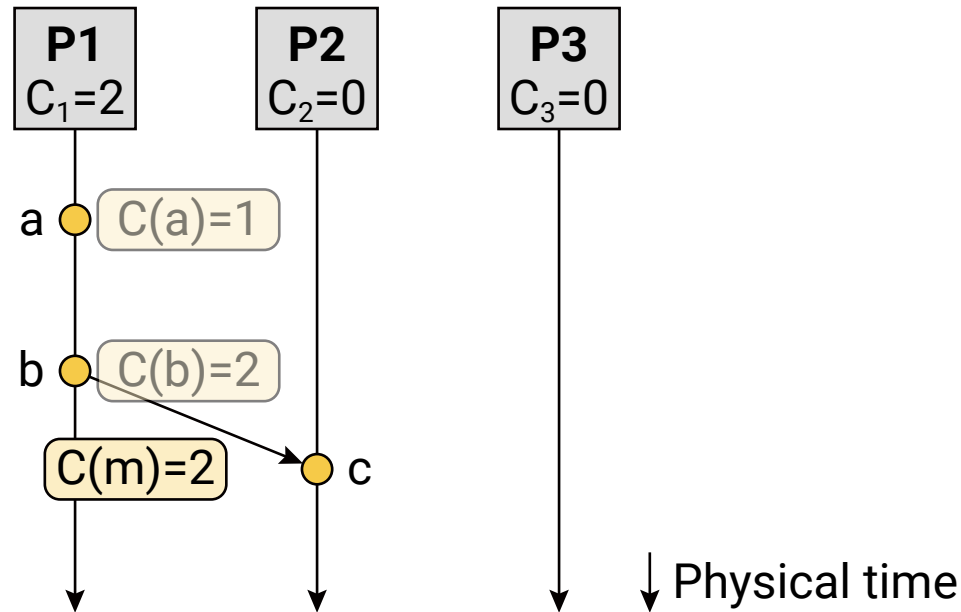
The Lamport Clock Algorithm

- Each process P_i maintains a local clock C_i
- Before executing an event, $C_i \leftarrow C_i + 1$
 - Send the local clock in the message m



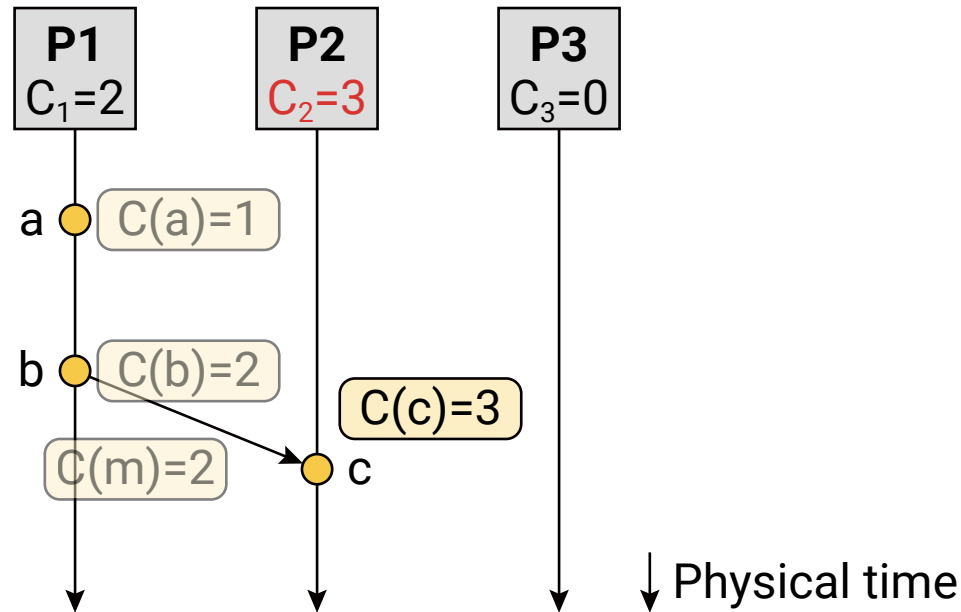
The Lamport Clock Algorithm

3. On process P_j receiving a message m :
- Set C_j and receive event time $C(c) \leftarrow 1 + \max\{C_j, C(m)\}$



The Lamport Clock Algorithm

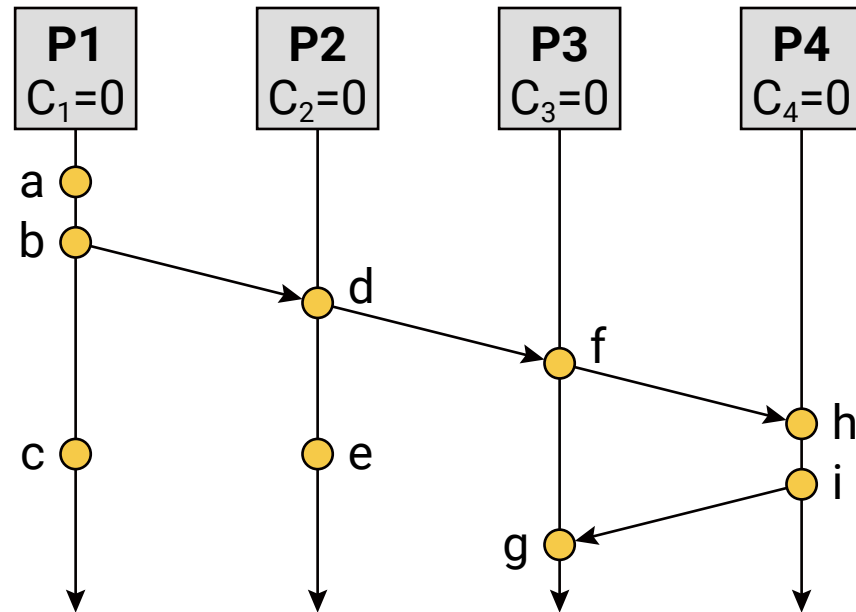
3. On process P_j receiving a message m :
- Set C_j and receive event time $C(c) \leftarrow 1 + \max\{C_j, C(m)\}$



Lamport Timestamps: Ordering All Events

- Break ties by appending the process number to each event:
 1. Process P_i timestamps event e with $C_{i(e)}.i$
 2. $C(a).i < C(b).j$ when:
 - $C(a) < C(b)$, or $C(a) = C(b)$ and $i < j$
- Now, for any two events a and b , $C(a) < C(b)$ or $C(b) < C(a)$
 - This is called a total ordering of events

Order All These Events



Take-away Points: Lamport Clocks

- Can totally-order events in a distributed system: that's useful!
 - We will see an application of Lamport clocks for totally-ordered multicast later

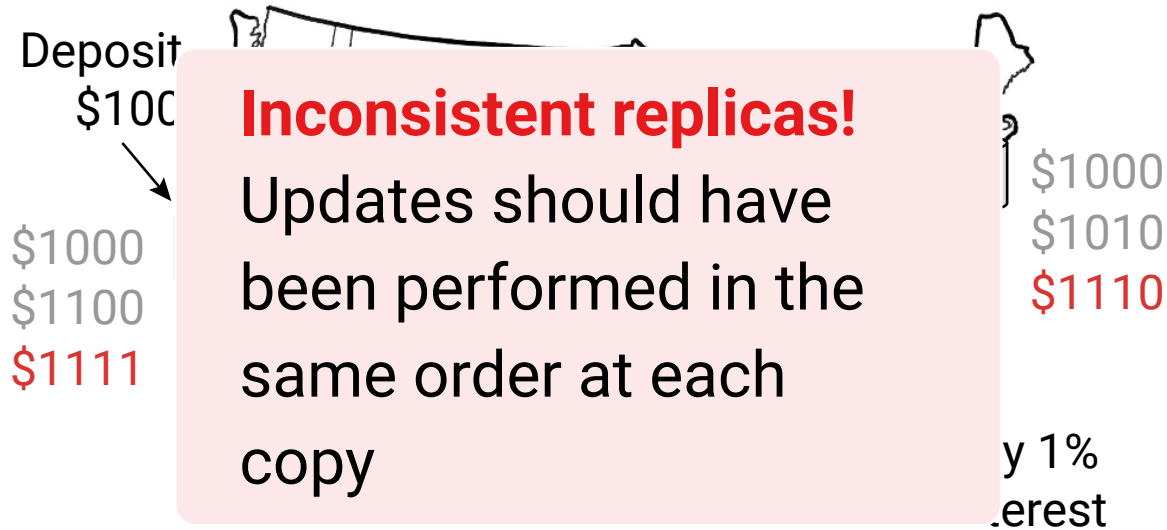
Take-away Points: Lamport Clocks

- Can totally-order events in a distributed system: that's useful!
 - We will see an application of Lamport clocks for totally-ordered multicast later
- But: while by construction, $a \rightarrow b$ implies $C(a) < C(b)$,
 - The converse is not necessarily true: $C(a) < C(b)$ does not naturally imply $a \rightarrow b$
 - $C(a) < C(b)$ can also occur when $a \parallel b$, provided that $a.i < b.j$
 - (But, if $C(a) < C(b)$, then $b \rightarrow a$ cannot be true)

Can't use Lamport timestamps to infer **causal relationships** between events

Multi-site Database Inconsistency

- Replicate the database, keep one copy in SF, one in NYC
 - Client sends reads to the nearest copy
 - Client sends update to both copies



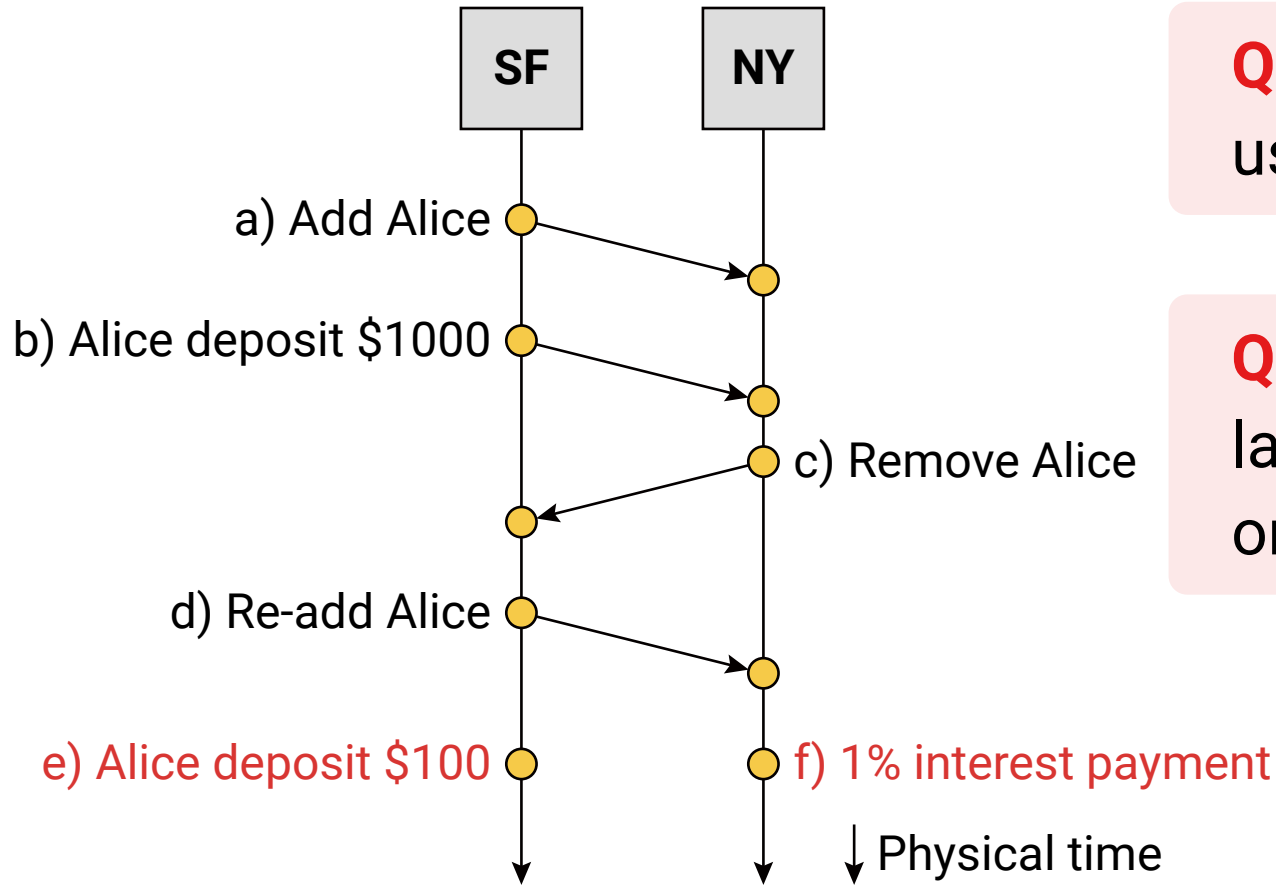
Totally-Ordered Multicast



Goal All sites apply updates in (same) Lamport clock order

- Client sends update to one replica site j
 - Replica assigns it Lamport timestamp $C_j.j$
- Key idea: Place events into a sorted **local queue**
 - **Sorted** by increasing Lamport timestamps

Totally-Ordered Multicast



Q1 What is bad about using order a, b, d, c ?

Q2 What are all the valid lamport timestamp total orders of $a-f$?



Totally-Ordered Multicast (Almost correct)



1. On receiving an update from client, broadcast to others (including yourself)
 - (Node -> node communication is FIFO and asynchronous)



Totally-Ordered Multicast (Almost correct)



1. On receiving an update from client, broadcast to others (including yourself)
 - (Node -> node communication is FIFO and asynchronous)
2. On receiving an update from replica:
 1. Add it to your local queue
 2. Broadcast an **acknowledgement message** to every replica (including yourself)



Totally-Ordered Multicast (Almost correct)



1. On receiving an update from client, broadcast to others (including yourself)
 - (Node -> node communication is FIFO and asynchronous)
2. On receiving an update from replica:
 1. Add it to your local queue
 2. Broadcast an **acknowledgement message** to every replica (including yourself)
3. On receiving an acknowledgement:
 - Mark corresponding update **acknowledged** in your queue



Totally-Ordered Multicast (Almost correct)



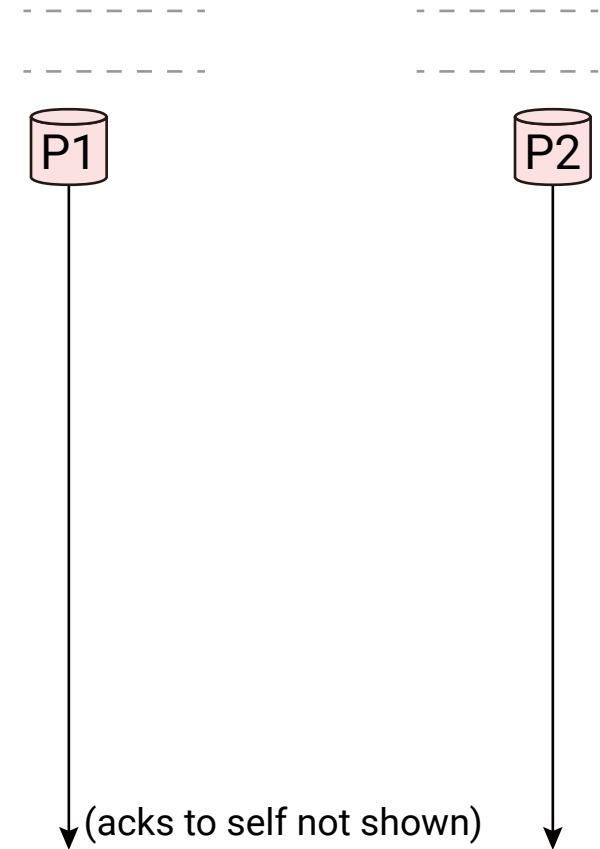
1. On receiving an update from client, broadcast to others (including yourself)
 - (Node -> node communication is FIFO and asynchronous)
2. On receiving an update from replica:
 1. Add it to your local queue
 2. Broadcast an **acknowledgement message** to every replica (including yourself)
3. On receiving an acknowledgement:
 - Mark corresponding update **acknowledged** in your queue
4. **Remove and process** updates **everyone** has ack'ed from **head** of queue



Totally-Ordered Multicast (Almost correct)

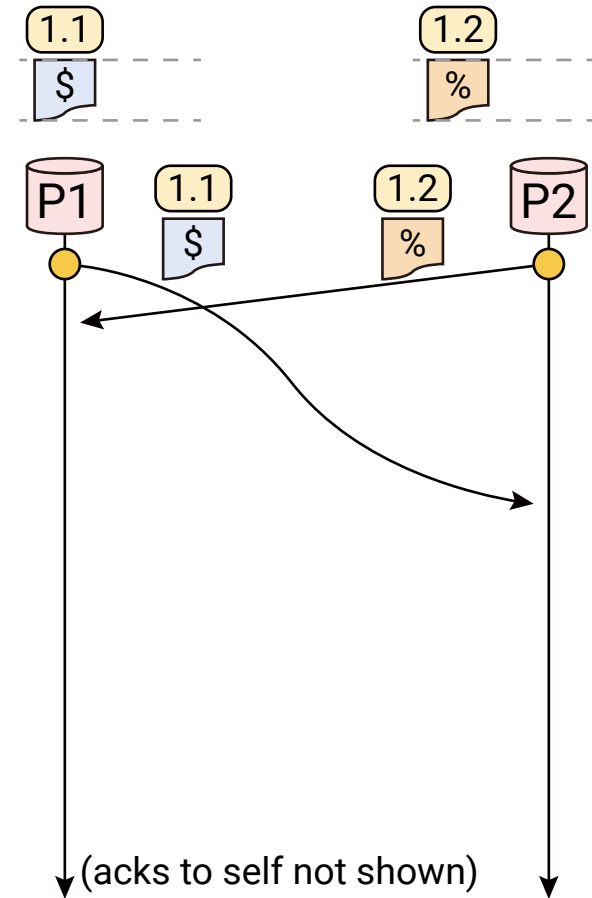
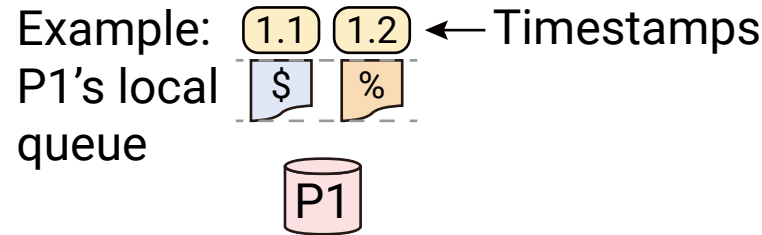


0. Both database starts with an empty queue



Totally-Ordered Multicast (Almost correct)

0. Both database starts with an empty queue
1. P1 queues \$, P2 queues %

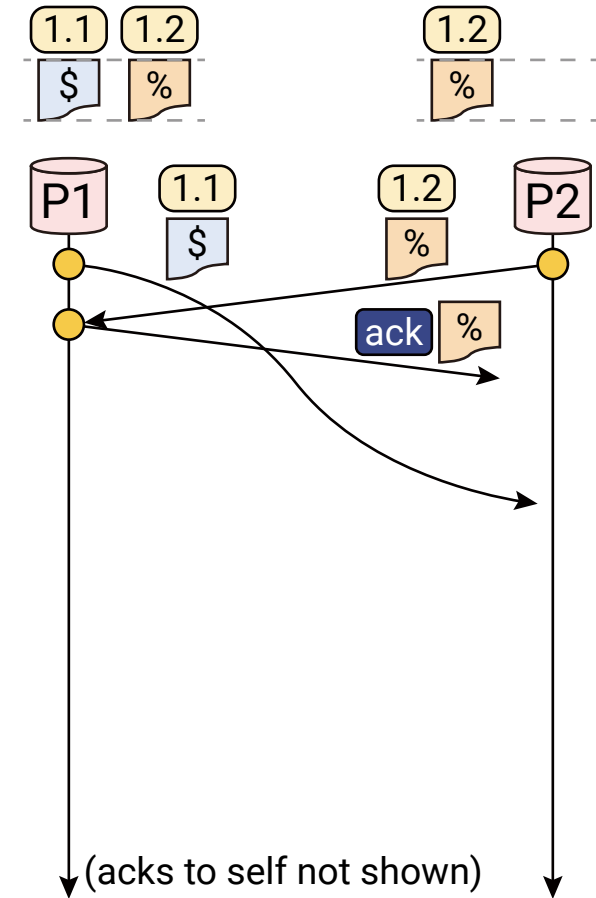




Totally-Ordered Multicast (Almost correct)



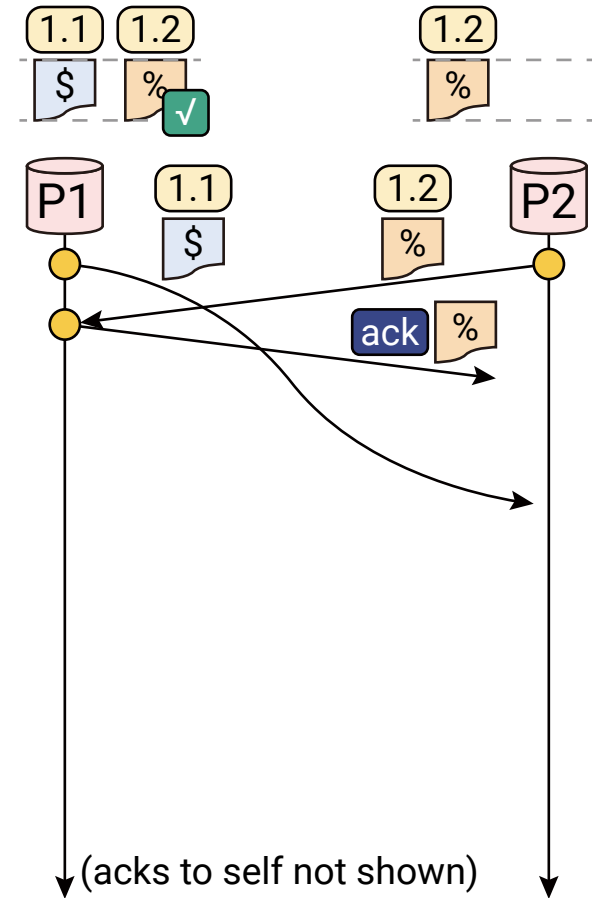
0. Both database starts with an empty queue
1. P1 queues \$, P2 queues %
2. P1 queues % and **ack's %** to itself and P2



Totally-Ordered Multicast (Almost correct)

0. Both database starts with an empty queue
1. P1 queues \$, P2 queues %
2. P1 queues % and **ack's %** to itself and P2

P1 marks % fully ack'ed

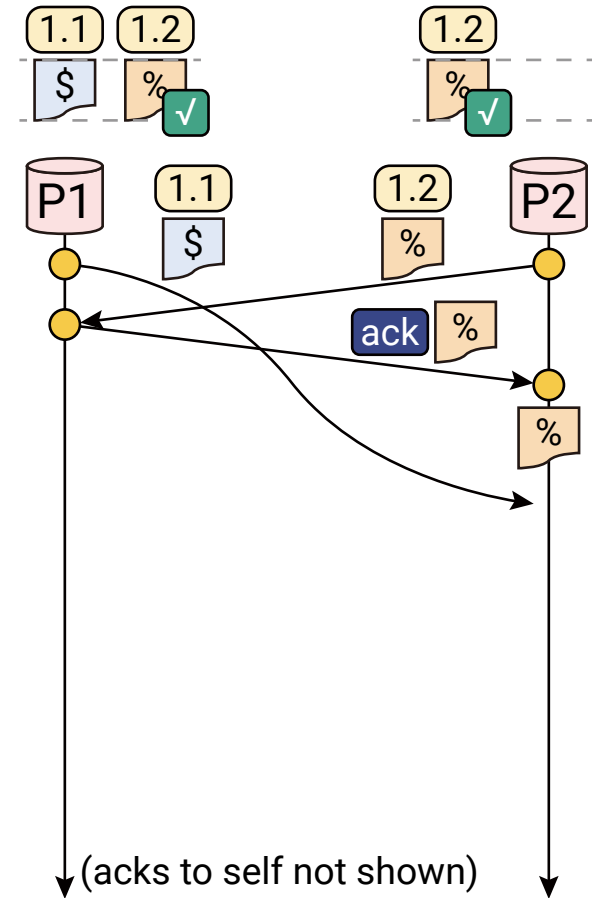


Totally-Ordered Multicast (Almost correct)

0. Both database starts with an empty queue
1. P1 queues \$, P2 queues %
2. P1 queues % and **ack's %** to itself and P2
3. P2 marks % fully ack'ed

P1 marks % fully ack'ed

P2 processes %

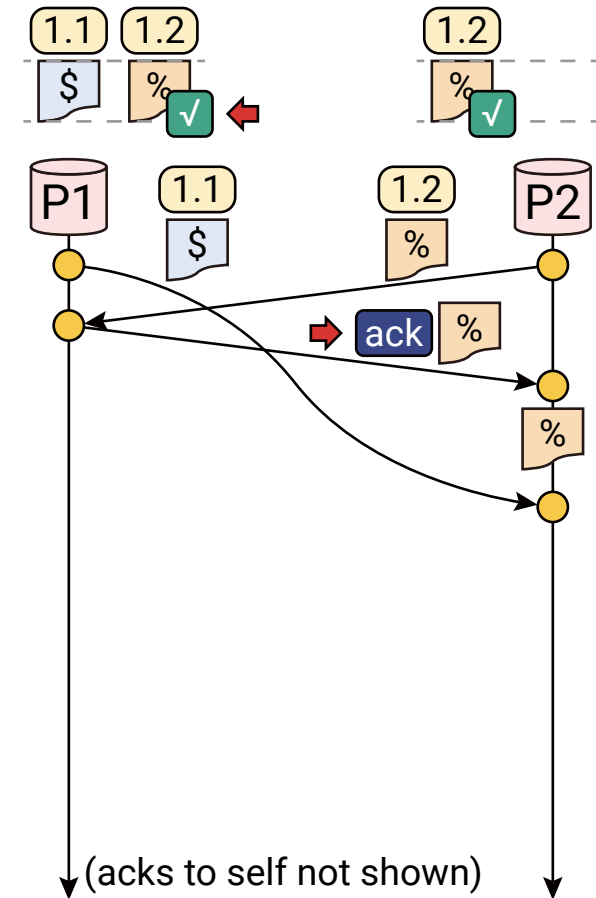


Totally-Ordered Multicast (Almost correct)

0. Both database starts with an empty queue
1. P1 queues \$, P2 queues %
2. P1 queues % and **ack's %** to itself and P2
3. P2 marks % fully ack'ed

P1 marks % fully ack'ed

P2 processes %





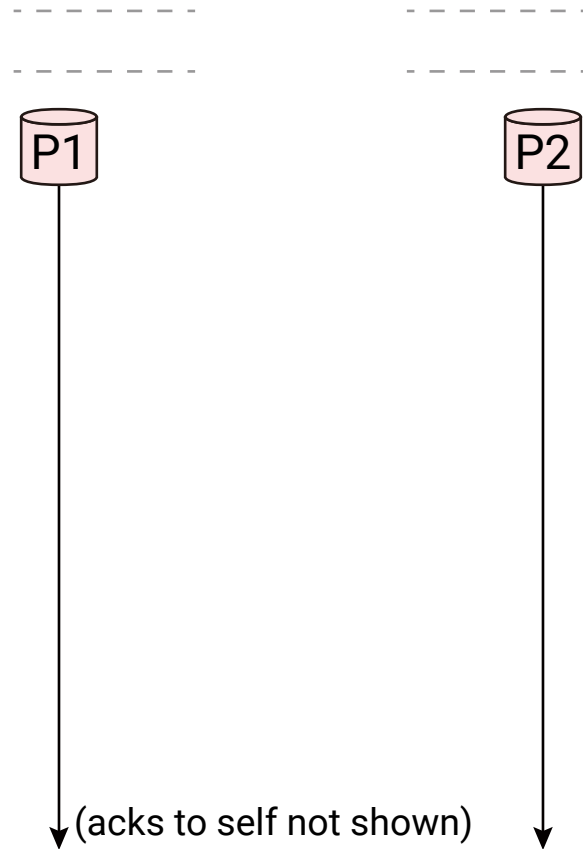
Totally-Ordered Multicast (Correct version)



1. On receiving an update from client, broadcast to others (including yourself)
2. On receiving **or processing** an update:
 1. Add it to your local queue, **if received update**
 2. Broadcast an acknowledgement message to every replica (including yourself), **only from head of queue**
3. On receiving an acknowledgement:
 - Mark corresponding update acknowledged in your queue
4. Remove and process updates everyone has ack'ed from head of queue

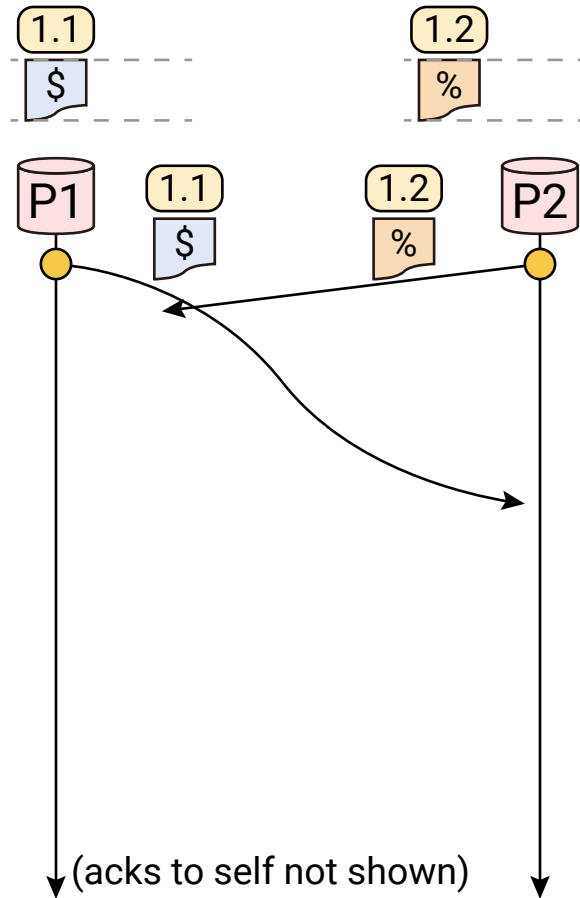


Totally-Ordered Multicast (Correct version)



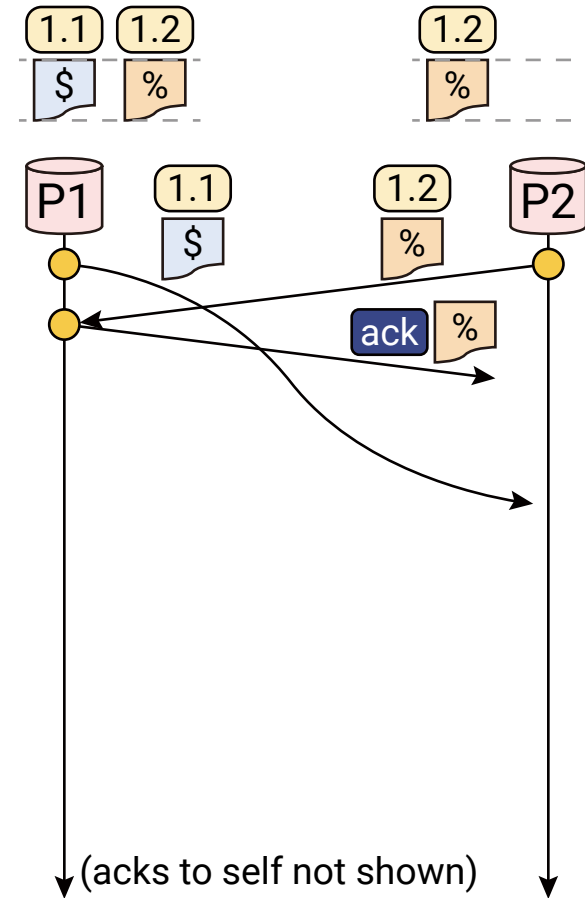
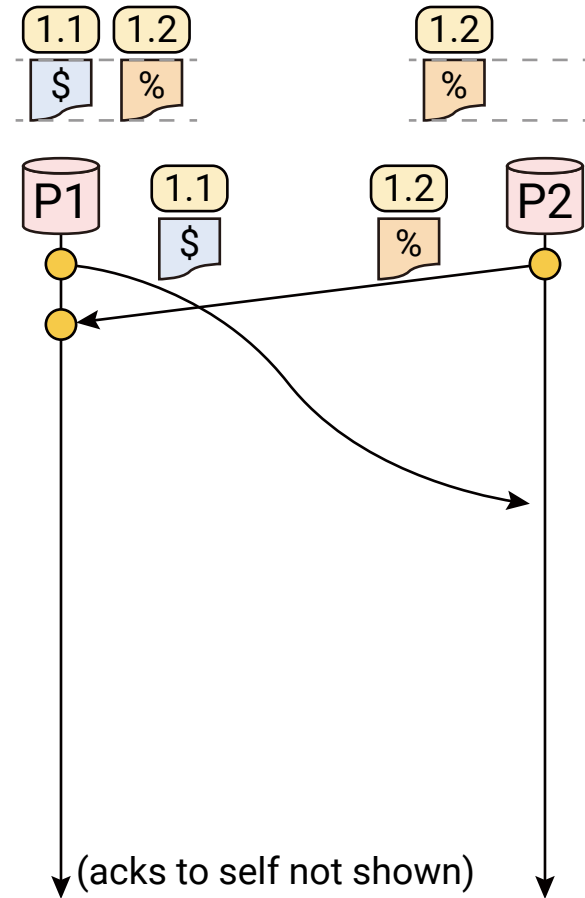


Totally-Ordered Multicast (Correct version)



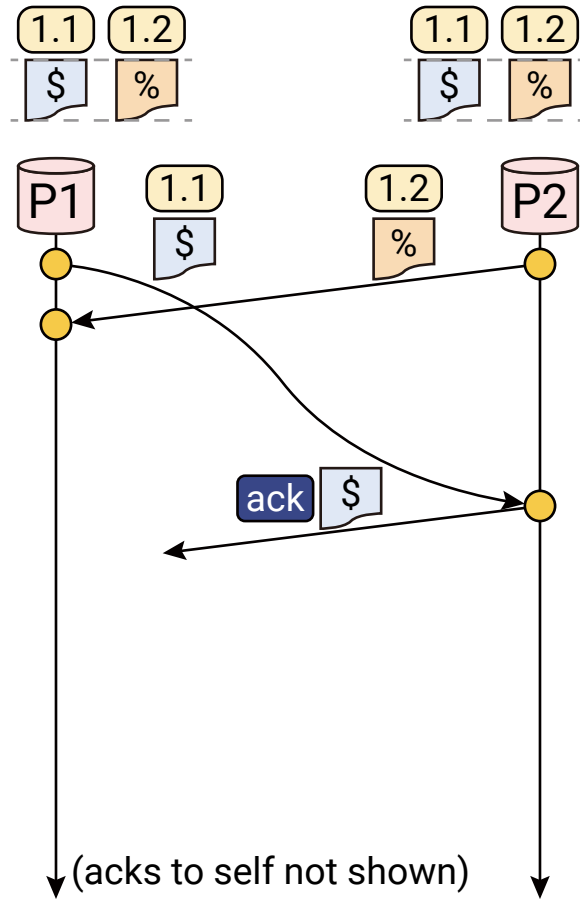


Totally-Ordered Multicast (Correct version)



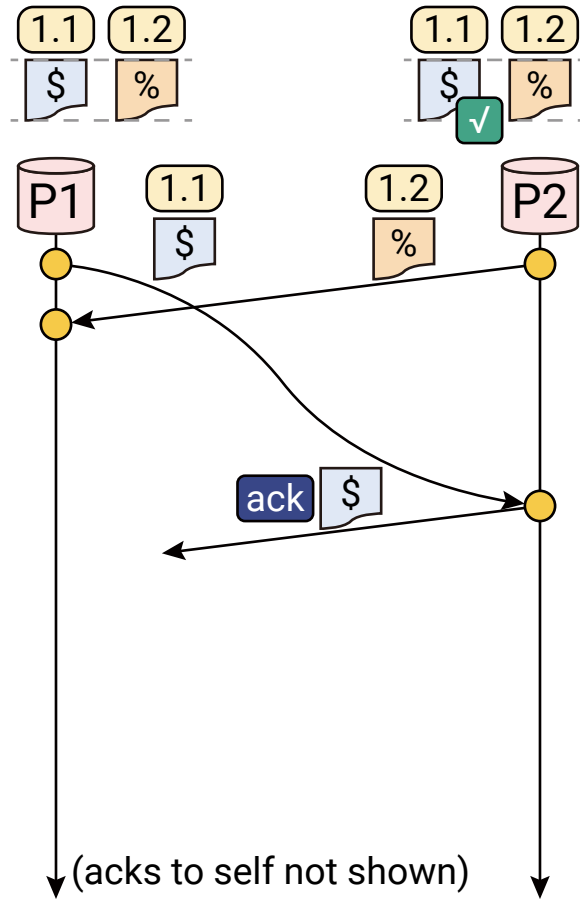


Totally-Ordered Multicast (Correct version)



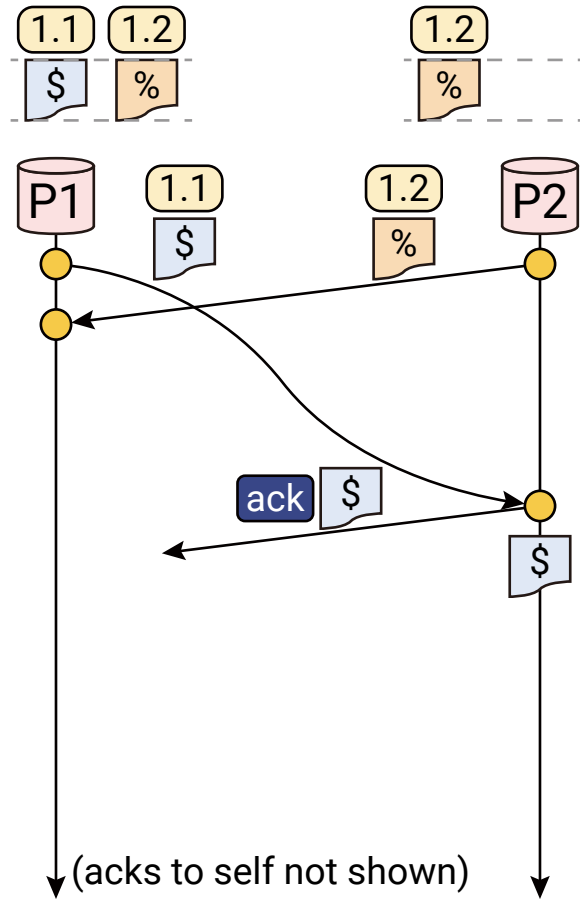


Totally-Ordered Multicast (Correct version)



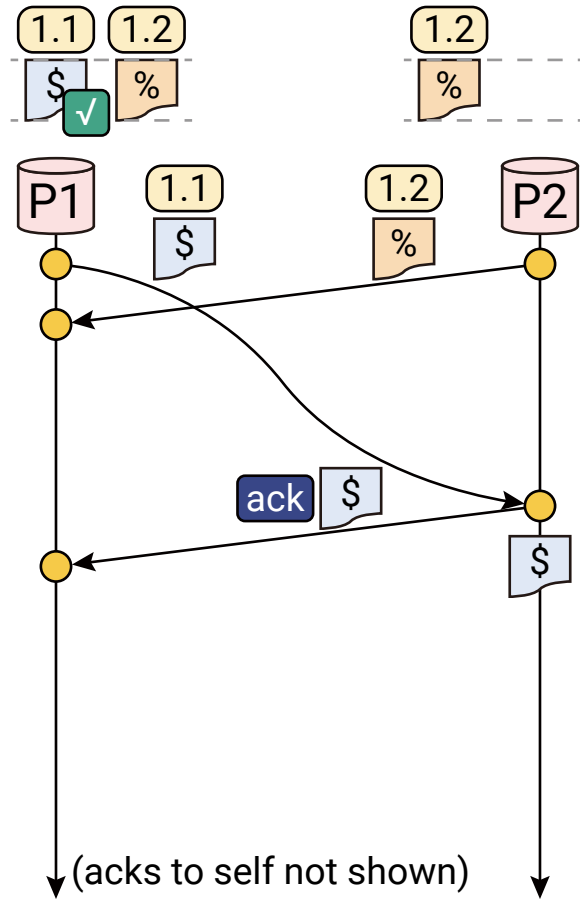


Totally-Ordered Multicast (Correct version)



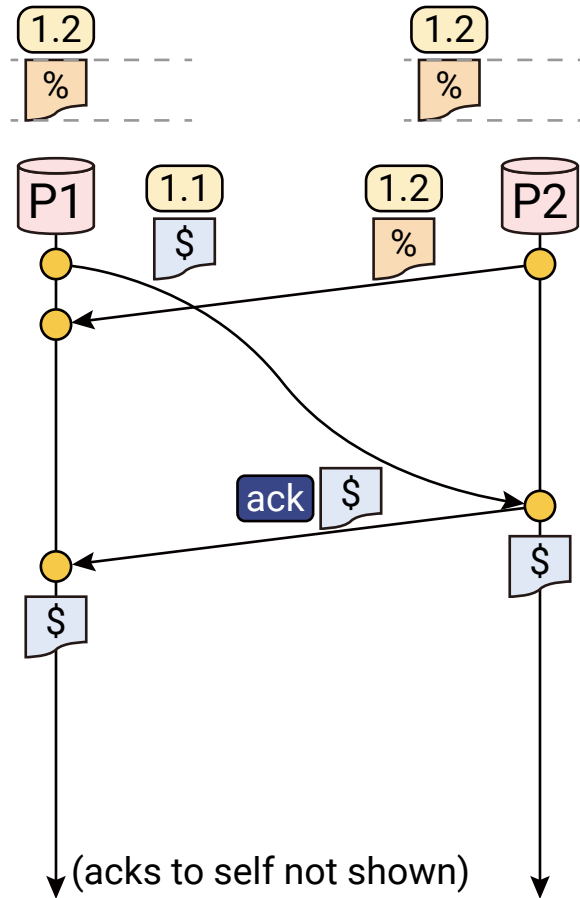


Totally-Ordered Multicast (Correct version)



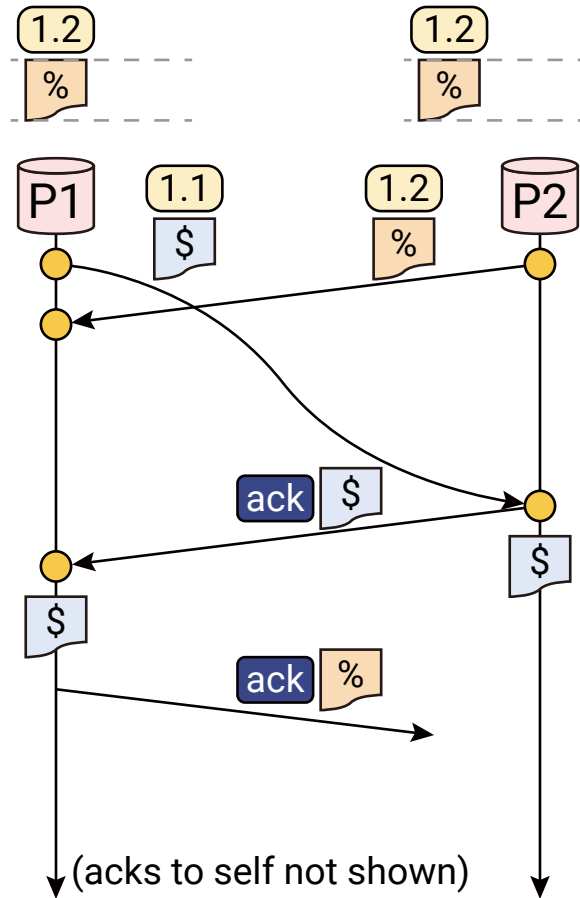


Totally-Ordered Multicast (Correct version)



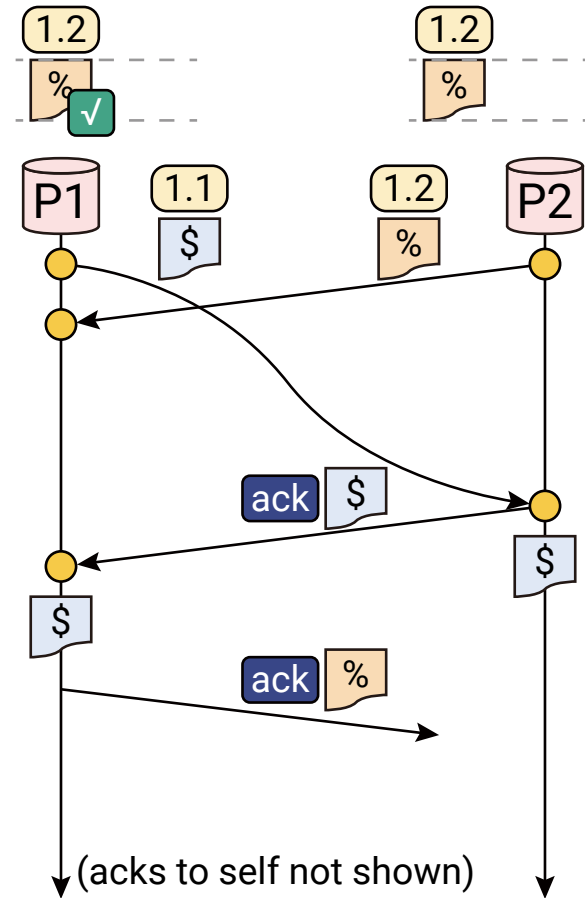


Totally-Ordered Multicast (Correct version)



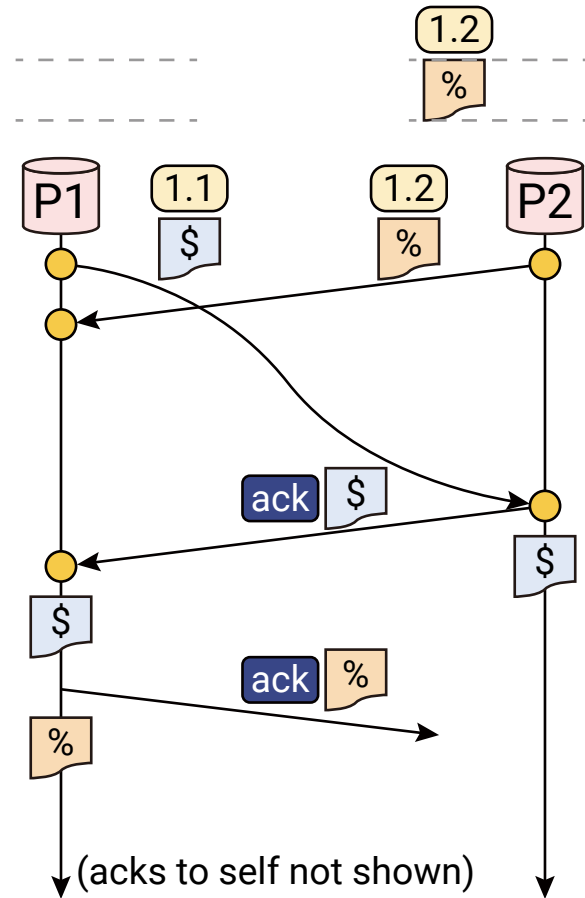


Totally-Ordered Multicast (Correct version)



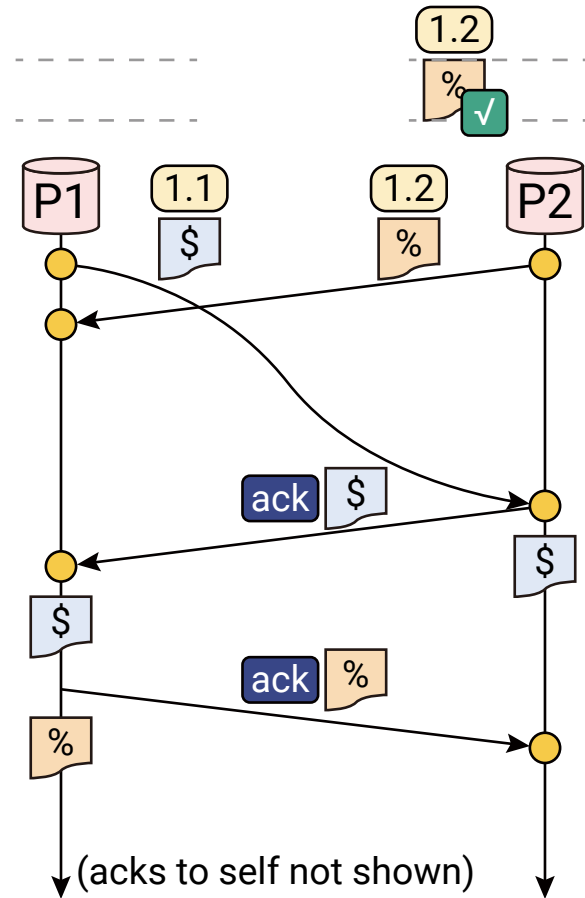


Totally-Ordered Multicast (Correct version)



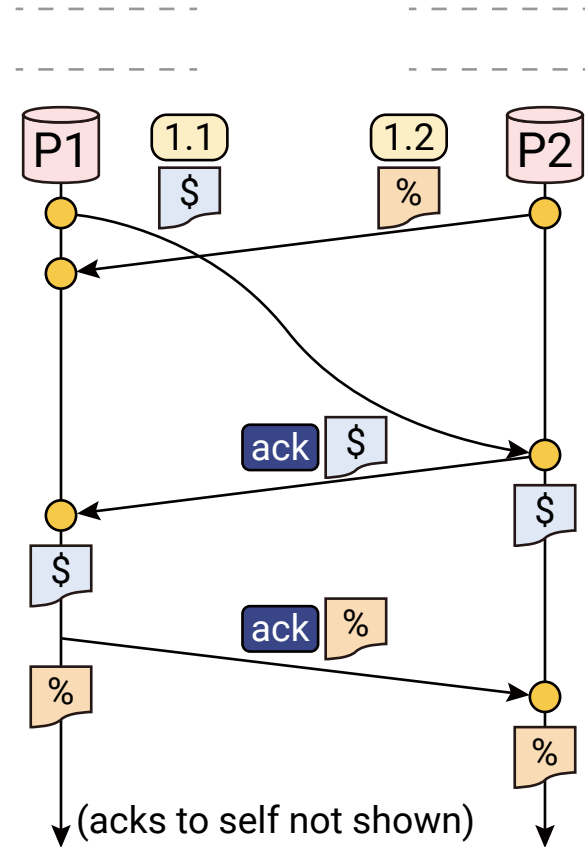


Totally-Ordered Multicast (Correct version)





Totally-Ordered Multicast (Correct version)



So, Are We Done?



- Does totally-ordered multicast solve the problem of multi-site replication in general?
 - Not by a long shot!

So, Are We Done?



- Does totally-ordered multicast solve the problem of multi-site replication in general?
 - Not by a long shot!
- 1. Our protocol **assumed**:
 - No node failures
 - No message loss
 - No message corruption

So, Are We Done?



- Does totally-ordered multicast solve the problem of multi-site replication in general?
 - Not by a long shot!
- 1. Our protocol **assumed**:
 - No node failures
 - No message loss
 - No message corruption
- 2. All to all communication **does not scale**
- 3. **Waits forever** for message delays (performance?)

Lamport Clocks Review



- $Q: a \rightarrow b \Rightarrow$

Lamport Clocks Review



- $Q: a \rightarrow b \Rightarrow C(a) < C(b)$
- $Q: C(a) < C(b) \Rightarrow$

Lamport Clocks Review

- Q: $a \rightarrow b \Rightarrow C(a) < C(b)$
- Q: $C(a) < C(b) \Rightarrow b \nrightarrow a$ ($a \rightarrow b$ or $a \parallel b$)
- Q: $a \parallel b \Rightarrow$

Lamport Clocks Review

- Q: $a \rightarrow b \Rightarrow C(a) < C(b)$
- Q: $C(a) < C(b) \Rightarrow b \nrightarrow a$ ($a \rightarrow b$ or $a \parallel b$)
- Q: $a \parallel b \Rightarrow$ nothing

Lamport Clocks Review

- Q: $a \rightarrow b \Rightarrow C(a) < C(b)$
- Q: $C(a) < C(b) \Rightarrow b \nrightarrow a$ ($a \rightarrow b$ or $a \parallel b$)
- Q: $a \parallel b \Rightarrow$ nothing

Lamport clock timestamps do not capture causality

- Given two timestamps $C(a)$ and $C(z)$, want to know whether there's a chain of events linking them:
 - $a \rightarrow b \rightarrow \dots \rightarrow y \rightarrow z$

Vector Clock: Introduction

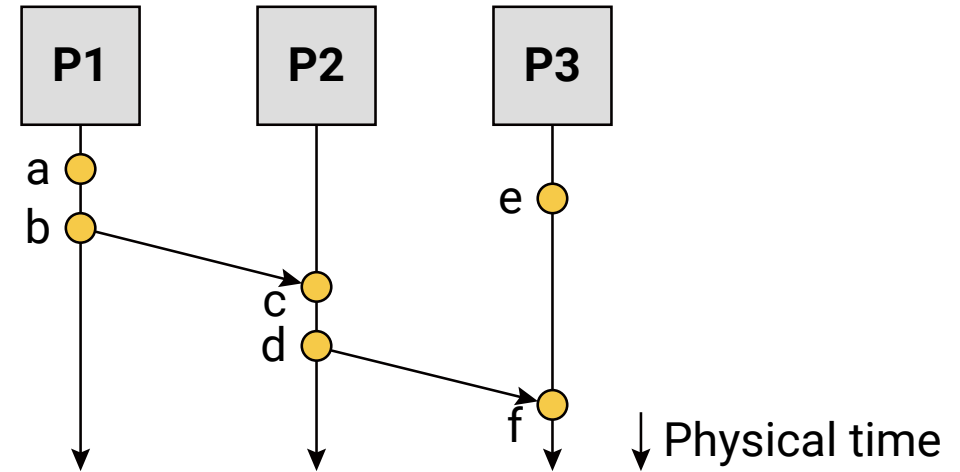
- One integer can't order events in more than one process
- So, a **Vector Clock (VC)** is a vector of integers, one entry for each process in the entire distributed system
 - Label event e with $VC(e) = [c_1, c_2, \dots, c_n]$
 - Each entry c_k is a count of events in process k that causally precede e

Vector Clock: Update Rules

- Initially, all vectors are $[0, 0, \dots, 0]$
- Two update rules:
 1. For each local event on process i , increment local entry c_i
 2. If process j receives message with vector $[d_1, d_2, \dots, d_n]$:
 - Set each local entry $c_k \leftarrow \max\{c_k, d_k\}$
 - Increment local entry c_j

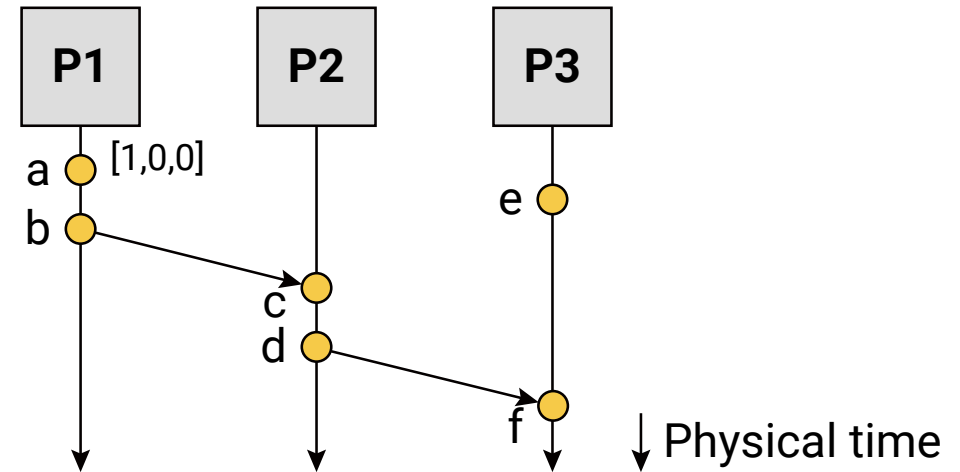
Vector Clock: Example

- All processes' VCs start at $[0, 0, 0]$



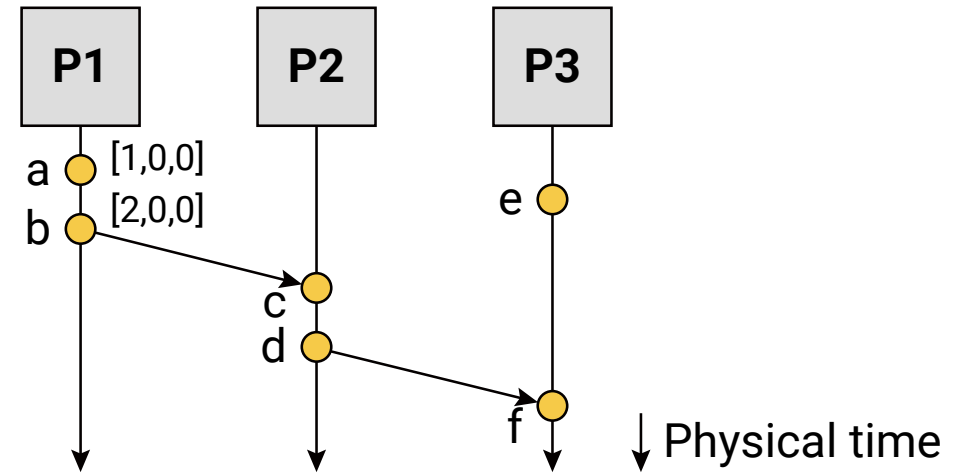
Vector Clock: Example

- All processes' VCs start at $[0, 0, 0]$
- Applying local update rule



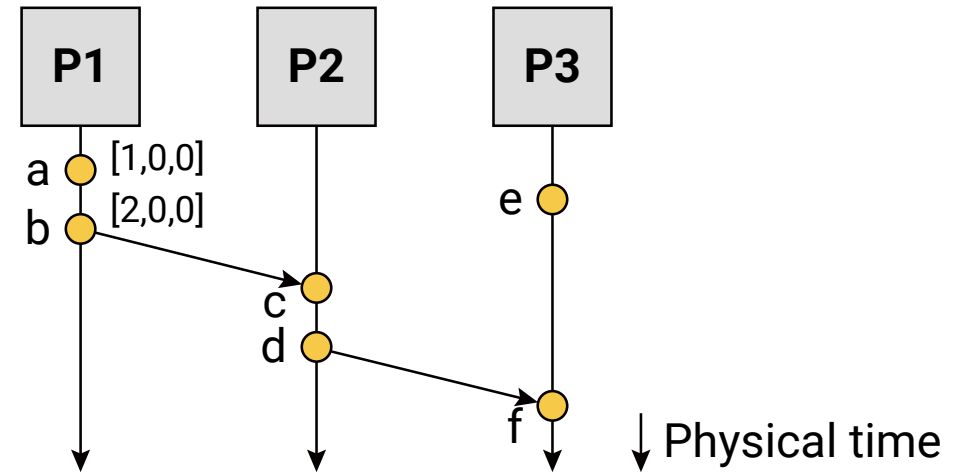
Vector Clock: Example

- All processes' VCs start at $[0, 0, 0]$
- Applying local update rule



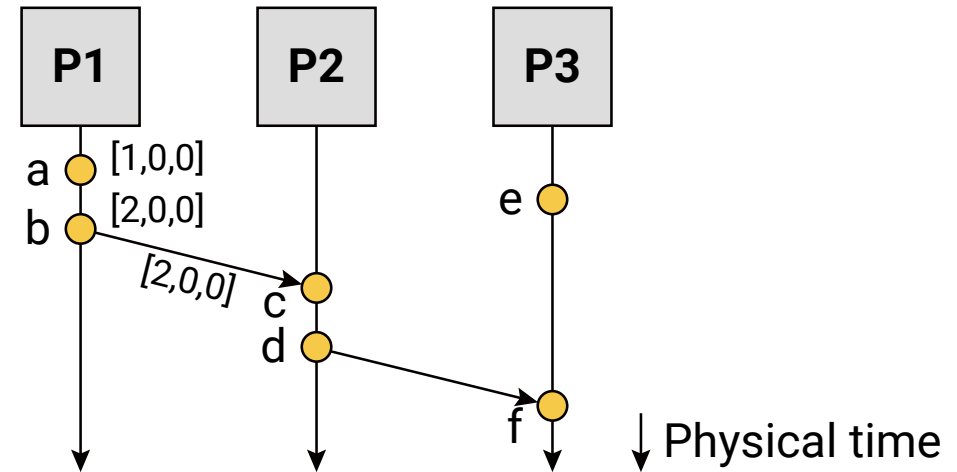
Vector Clock: Example

- All processes' VCs start at $[0, 0, 0]$
- Applying local update rule
- Applying message rule
 - Local vector clock piggybacks on inter-process messages



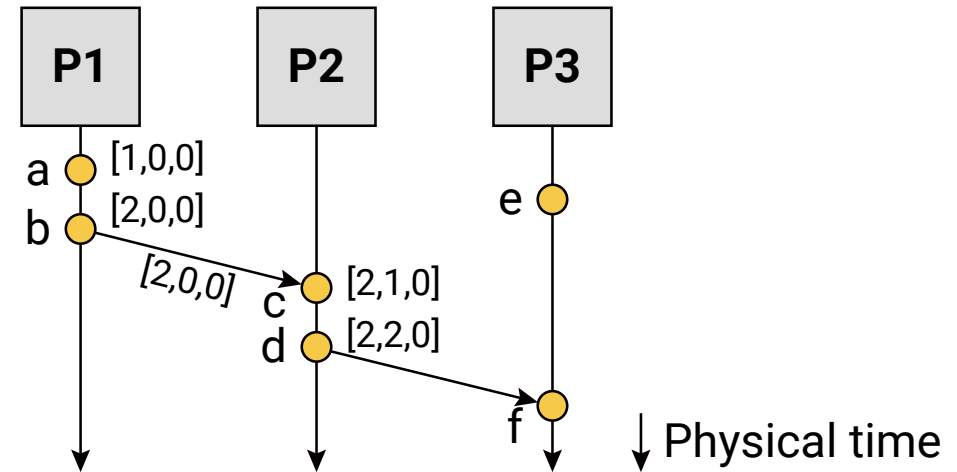
Vector Clock: Example

- All processes' VCs start at $[0, 0, 0]$
- Applying local update rule
- Applying message rule
 - Local vector clock piggybacks on inter-process messages



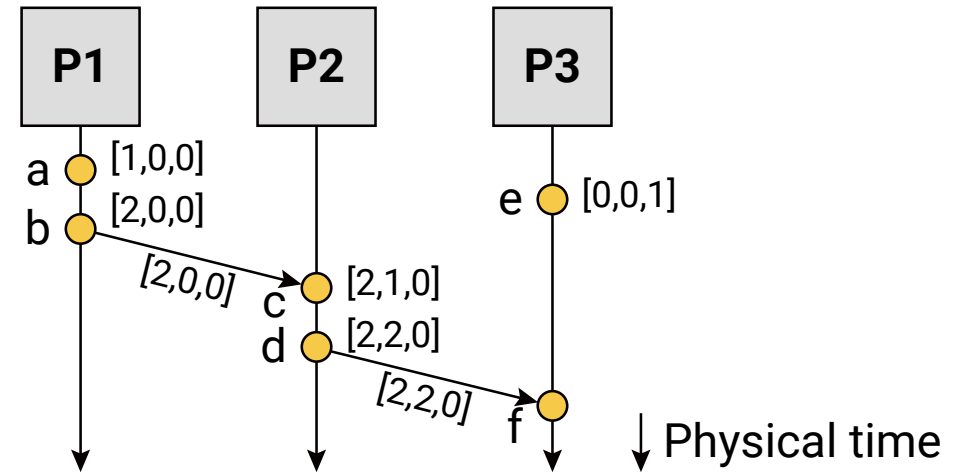
Vector Clock: Example

- All processes' VCs start at $[0, 0, 0]$
- Applying local update rule
- Applying message rule
 - Local vector clock piggybacks on inter-process messages



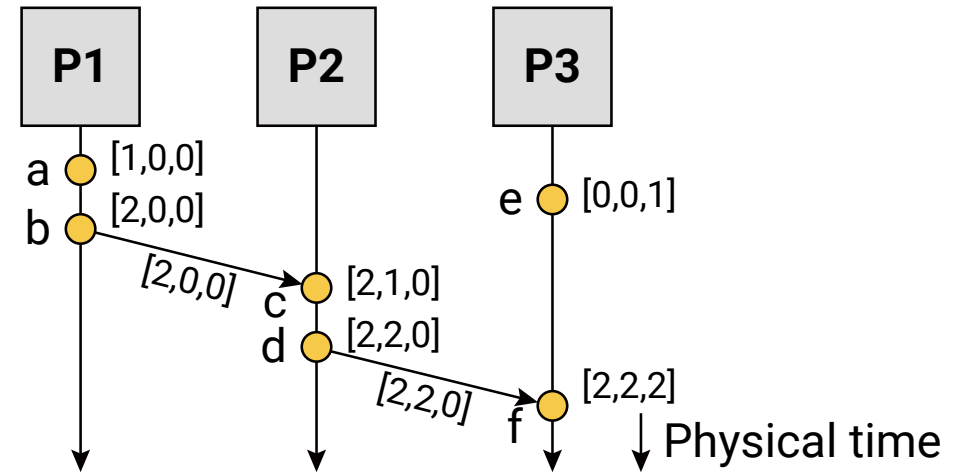
Vector Clock: Example

- All processes' VCs start at $[0, 0, 0]$
- Applying local update rule
- Applying message rule
 - Local vector clock piggybacks on inter-process messages



Vector Clock: Example

- All processes' VCs start at $[0, 0, 0]$
- Applying local update rule
- Applying message rule
 - Local vector clock piggybacks on inter-process messages



▶ Comparing Vector Timestamps

- Rule for comparing vector timestamps:
 - ▶ $V(a) = V(b)$ when $a_k = b_k$ for all k
 - ▶ $V(a) < V(b)$ when $a_k \leq b_k$ for all k and $V(a) \neq V(b)$
- Concurrency:
 - ▶ $V(a) \parallel V(b)$ if $a_i < b_i$ and $a_j > b_j$, some i and j

Vector Clocks Capture Causality

- $V(w) < V(z)$ then there is a chain of events linked by $\rightarrow$ between w and z
 - $V(a) \parallel V(w)$ then there is no such chain of events between a and w

▶ Comparing Vector Timestamps

- Rule for comparing vector timestamps:
 - ▶ $V(a) = V(b)$ when $a_k = b_k$ for all k
 - They are the same event
 - ▶ $V(a) < V(b)$ when $a_k \leq b_k$ for all k and $V(a) \neq V(b)$
 - $a \rightarrow b$
- Concurrency:
 - ▶ $V(a) \parallel V(b)$ if $a_i < b_i$ and $a_j > b_j$, some i and j
 - $a \parallel b$

© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.